Notes of Deep Reinforcement Learning

Gralerfics

November 02, 2026

Contents

1. Fundamentals 4
   1.1. Markov Chains 4
   1.2. Return and Value Functions 6
   1.3. Bellman Expectation Equations 8
   1.4. Optimal Bellman Equations 11
2. Value-Based Learning 14
   2.1. Policy Iteration and Value Iteration 14
   2.2. Tabular Q-Learning and SARSA 14
   2.3. Value Approximation 16
      2.3.1. Dataset 16
      2.3.2. Residual Value Gradients 17
      2.3.3. Value Semi-gradients 19
      2.3.4. Online Q-Learning 19
   2.4. Stabilization Issues and Techniques 20
      2.4.1. Experience Replay 20
      2.4.2. Exploration 22
      2.4.3. Target Networks 22
   2.5. Deep Q-Networks (DQN) 23
      2.5.1. Why “Deep” Learning 24
      2.5.2. Q-Learning extensions 24
         2.5.2.1. Double Q-Learning 24
         2.5.2.2. Dueling Network Architectures 24
      2.5.3. Partial Observability and Deep Recurrent Q-Networks (DRQN) 25
3. Policy Optimization and On-Policy Actor-Critic 29
   3.1. Parameterized Stochastic Policies 29
   3.2. Policy Gradient Theorem 30
      3.2.1. REINFORCE 32
   3.3. Actor-Critic Algorithms 32
      3.3.1. Value Function Estimation 33
      3.3.2. Advantage Actor-Critic (A2C) 35
      3.3.3. Asynchronous Advantage Actor-Critic (A3C) 38
      3.3.4. Convergence of Actor-Critic Methods 38
      3.3.5. Continuous Control 38
      3.3.6. Conclusion 39
   3.4. Trust Region Methods 39
      3.4.1. Off-Policy Gradients 39
      3.4.2. Trust-Region Policy Optimization (TRPO) 42
      3.4.3. Proximal Policy Optimization (PPO) 42
      3.4.4. Group Relative Policy Optimization (GRPO) 43
4. Off-Policy Actor-Critic 45
   4.1. Deterministic Policy Gradients (DPG) 45
      4.1.1. Deep DPG (DDPG) 45
      4.1.2. Twin Delayed DDPG (TD3) 46

4.2. Robust Decision Making 47
4.2.1. Regularization 47
4.2.2. Robust Reinforcement Learning 47
4.2.3. Planning as Inference 47
4.2.4. Stochastic Policy Gradients (SPG) and Deterministic Policy Gradients (DPG) 47
4.2.5. Reparameterization Trick 47
4.3. Maximum Entropy RL 47
4.3.1. Soft Actor-Critic (SAC) 47
5. Exploration 48
5.1. Uncertainty 49
5.2. Thompson Sampling 49
5.3. Optimistic Exploration 49
6. Offline Reinforcement Learning 50
7. Multi-Agent Reinforcement Learning 51
8. Multi-Task Reinforcement Learning 52
9. Model-Based Reinforcement Learning 53

1. Fundamentals

1.1. Markov Chains

我们将智能体的行为建模为一个**随机策略**（stochastic policy）$\pi(a \mid s)$，即在状态为 s 的条件下采取动作 a 的概率分布。**状态**（state）是一种对当前已知信息的描述，例如智能体所处环境、内部状态等；**动作**（action）则是智能体可选的行动，采取后可能导致状态发生变化，转移到另一个状态，即**状态转移**（state transition）。

在强化学习的背景下，我们会对“在某状态下采取某动作”的行为发放**奖励**（reward）以引导智能体的学习。奖励的发放也可以选取其他时机，例如采取动作后根据实际到达的新状态发放奖励等，不同约定互相之间通常都可转换，只是定义和符号差别。此处我们约定，某时刻的奖励仅为该时刻状态和动作的函数。

随时间发展状态不断转移，智能体和环境将演化出一条由状态、动作和奖励构成的随机序列：

$$\mathrm{T}_n = \{S_0, A_0, R_0, S_1, A_1, R_1, ..., S_{n-1}, A_{n-1}, R_{n-1}, S_n\} \tag{1}$$

序列的各个元素为随机变量，这是由于我们的策略 π 和状态转移模型 P 都将使用随机分布建模。严格来说，由前面的约定，我们采用的奖励函数是关于状态和动作的确定（deterministic）函数，故 R_t 并不是随机变量（或者视为只有一种概率为 1 的取值）。如果需要，奖励函数自然也可以用概率模型建模，用以刻画奖励的随机性；但奖励终究是人为设计的，此举通常没有特别的必要。

从该随机序列进行一次采样即得到一条**轨迹**（trajectory）：

$$\tau_n \coloneqq \{s_t, a_t, r_t\}_{t=0}^{n-1} \cup \{s_n\} \tag{2}$$

用小写字母表示样本以作区分。n 为时域窗口的长度，可以有 $n \to \infty$。s_n 为终端（terminal）状态，可以用“在 n 时刻后不再有奖励”来表达，即 $r_t = 0, \forall t \geq n$。

如果状态转移概率 P 仅和当前时刻的状态和动作有关，即满足马尔可夫性：

$$\begin{aligned} &\mathbb{P}(S_{t+1} = s \mid S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, ...) \\ =\ &\mathbb{P}(S_{t+1} = s \mid S_t = s_t, A_t = a_t), \quad \forall s \in \mathcal{S} \end{aligned} \tag{3}$$

则前述序列可视作一条**马尔可夫链**（Markov chain），其演化过程称为**马尔可夫决策过程**（Markov decision process，MDP），记为 $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \rho, P, r \rangle$，其中：

- $\mathcal{S}$ 为状态集合。
- $\mathcal{A}$ 为动作集合。
- $\rho(s)$ 为初始状态先验分布，即有 $S_0 \sim \rho(\cdot)$。
- $P(s' \mid s, a)$ 为状态转移概率，即有 $S_{t+1} \sim P(\cdot \mid s_t, a_t)$。
- $r(s, a)$ 为奖励函数，按我们的约定有 $R_t \equiv r_t = r(s_t, a_t)$。

每时刻采取的动作则由随机策略 $\pi(a \mid s)$ 描述，即有 $A_t \sim \pi(\cdot \mid s_t)$。

关于概率、条件概率和概率分布的符号约定：

我们通常用概率函数 $\mathbb{P}$，表达某个“事件”的概率。考虑一个随机变量 X 等于样本 x 的概率，X 不是事件，x 也不是事件，$X = x$ 才是一个事件，所以严格来说应该写为 $\mathbb{P}(X = x)$。如果需要表达条件概率，就将条件写到竖线符号的后面，注意条件本身也是事件，所以严格来说概率函数都要写成像 Equation 3 中的完整形式。

不过通常写完整会比较繁琐，所以可以适当简化。在 MDP 中我们定义的一系列随机变量或样本之间都有对应的下标，如 $S_t = s_t$ 等，于是有时直接简略地将 s_t 写到条件处，省去随机变量的标记；但如果是 s、s' 等不明确的符号，则建议写出完整事件。

注意区分概率函数 $\mathbb{P}$ 和我们在 MDP 中定义的转移概率函数 P，后者实际上和 ρ、π 一样，都是表示概率分布的函数，即概率质量函数（离散情况）或概率密度函数（连续情况）。

对于一个随机变量 X，我们可以用 $X \sim p(x)$ 表示其服从分布函数 $p(x) := \mathbb{P}(X = x)$，其中的 x 只是表示自变量的任意符号，所以有时也写成 $X \sim p(\cdot)$ 避免混淆，用点号表示该函数的自变量。在 MDP 中我们定义了形如 $S_{t+1} \sim P(\cdot \mid s_t, a_t)$、$\pi(\cdot \mid s_t)$ 的分布函数，里面存在用于表示条件竖线符号，但实际上这是容易混淆的。这些函数中的“条件”实际上表示“参数”，而参数按惯例最好用分号隔开，或者标记到上下标等地方。例如 S_{t+1} 服从的分布实际上是 $p(s) := P(s; s_t, a_t)$，只有 s 是自变量；$\pi(\cdot \mid s)$ 实际最好写为 $\pi(\cdot; s)$ 等。

不过由于竖线符号在各种材料中用得实在太多，这里我们就接受 $\pi(a \mid s)$ 等写法，但要注意不能混淆其本意。

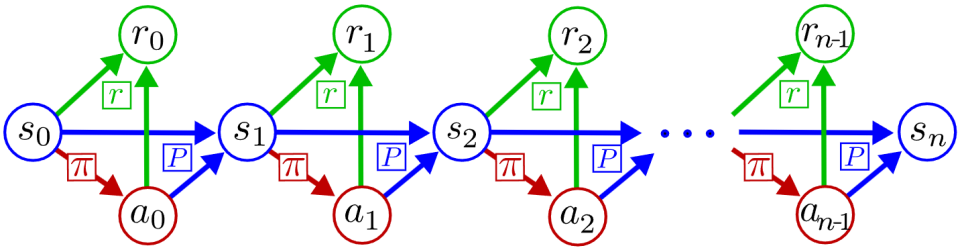


Figure 1: 马尔可夫链示意图（来自 TU Delft 课件）

总而言之，MDP 加上策略 π 这两个要素构成了马尔可夫链，如 Figure 1 所示。因此，随机轨迹序列 T 的分布参数依赖 MDP 参数和 π，可记为：

$$\mathrm{T} := \{S_0, A_0, R_0, ..., S_n\} \sim p_{\mathrm{T}}(\cdot; \mathcal{M}, \pi) \tag{4}$$

如前约定，我们用分号表达“参数”的含义。严格来说，n 也应该写在 MDP 参数里，不过我们主要考虑 $n \to \infty$，通常就不写了。

在我们的讨论范畴内通常默认 MDP 模型（部分证明涉及给定模型下不同策略的比较），所以可以只标记 π 用以区分，类似地还有之后的 $\mathbb{E}_\pi$、V^π 等记号，故之后会出现如下形式表达：

$$\mathrm{T} \sim p_{\mathrm{T}}^{\pi}(\cdot) \tag{5}$$

且为了简便起见，我们还约定：

$$\mathbb{E}_\pi[\cdot] = \mathbb{E}_{\mathrm{T}\sim p_{\mathrm{T}}^{\pi}(\cdot)}[\cdot] \tag{6}$$

$$\mathbb{E}_\rho[\cdot] = \mathbb{E}_{S_0\sim \rho(\cdot)}[\cdot] \tag{7}$$

详细的关于期望与条件期望的符号约定详见 Section 1.3 中的某处注释。

1.2. Return and Value Functions

一条轨迹的总体奖励称为**回报**（return），通常我们会每时刻乘上**折扣因子**（discount）来刻画“长期奖励贡献递减”的特性，不考虑太远的未来，但也不能只看眼前的奖励，再考虑 $n \to \infty$，定义**折扣回报**（discounted return）：

$$J(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t \tag{8}$$

折扣回报是基于轨迹的，而轨迹是随机的，为了衡量一个状态的价值，我们需要的是回报的期望：

$$V^\pi(s) \coloneqq \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \,\middle|\, S_0 = s \right] \tag{9}$$

这个期望回报称为**值函数**（value function），表示在使用策略 π 的前提下从 s 出发获得的平均折扣回报，加了条件 $S_0 = s$ 表示对给定策略下某个状态平均价值的一种衡量。应用全概率公式引入初态先验分布 ρ 即可得到对整个策略平均价值的衡量：

$$J^\pi \coloneqq \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] = \mathbb{E}_\rho[V^\pi(S_0)] \tag{10}$$

为了更好地研究策略，我们有时还希望衡量在某状态下采取某动作的平均价值，故引入**动作值函数**（action-value function）：

$$Q^\pi(s,a) \coloneqq \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \,\middle|\, S_0 = s, A_0 = a \right] \tag{11}$$

由此可以看出 J^π、V^π 和 Q^π 在条件上是递进的关系，Equation 10 中描述了前两者的关系，对后两者则有全概率公式：

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{A\sim\pi(\cdot|s)}[Q^\pi(s,A)] \\ &= \sum_a \pi(a \mid s) Q^\pi(s,a) \end{aligned} \tag{12}$$

从定义出发可以得到逆过来的关系（证明见后）：

$$\begin{aligned} Q^\pi(s,a) &= r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot|s,a)}[V^\pi(S')] \\ &= r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^\pi(s') \end{aligned} \tag{13}$$

关于值函数与定义中初态时间索引无关的证明：

值函数的定义中是从 $S_0 = s$ 开始计算折扣回报，那由于马尔可夫性，它和从任意 $S_k = s$ 开始计算其后（前面的不算）的回报理应是一样的，即有：

$$\mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+k} \,\middle|\, S_k = s \right] = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \,\middle|\, S_0 = s \right] =: V^\pi(s) \tag{14}$$

只是时间轴平移了 k 步，直觉上很好理解，但还是需要形式上证明一下，后面会用到。为方便展开取前 n 步，左式为：

$$\begin{aligned} &\mathbb{E}_\pi \left[\sum_{t=0}^{n} \gamma^t R_{t+k} \,\middle|\, S_k = s \right] \\ &= \sum_{a_k, s_{k+1}, \ldots, a_{k+n}} \left(\sum_{t=0}^{n} \gamma^t r(s_{t+k}, a_{t+k}) \right) \prod_{t=0}^{n} \pi(a_{t+k} \mid s_{t+k}) \\ &\quad P(s_{t+k+1} \mid s_{t+k}, a_{t+k}), \quad s_k = s \end{aligned} \tag{15}$$

令 $\tilde{s}_t = s_{t+k}, \tilde{a}_t = a_{t+k}$ 则有 $\tilde{s}_0 = s_k = s$，于是上式化为：

$$\sum_{\tilde{a}_0, \tilde{s}_1, \ldots, \tilde{a}_n} \left(\sum_{t=0}^{n} \gamma^t r(\tilde{s}_t, \tilde{a}_t) \right) \prod_{t=0}^{n} \pi(\tilde{a}_t \mid \tilde{s}_t) P(\tilde{s}_{t+1} \mid \tilde{s}_t, \tilde{a}_t) \tag{16}$$

而另一方面，直接从 $s_0 = s$ 出发，右式为：

$$\begin{aligned} &\mathbb{E}_\pi \left[\sum_{t=0}^{n} \gamma^t R_t \,\middle|\, S_0 = s \right] \\ &= \sum_{a_0, s_1, \ldots, a_n} \left(\sum_{t=0}^{n} \gamma^t r(s_t, a_t) \right) \prod_{t=0}^{n} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t), \quad s_0 = s \end{aligned} \tag{17}$$

二者完全相同，只是符号不同。因此对任意 n 都有：

$$\mathbb{E}_\pi \left[\sum_{t=0}^{n} \gamma^t R_{t+k} \,\middle|\, S_k = s \right] = \mathbb{E}_\pi \left[\sum_{t=0}^{n} \gamma^t R_t \,\middle|\, S_0 = s \right] \tag{18}$$

令 $n \to \infty$，若 $0 \le \gamma < 1$ 且奖励有界，则折扣和收敛，则证得 Equation 14。

关于“动作值函数-值函数”递推关系的证明：

我们来证明 Equation 13。首先：

$$
\begin{aligned}
Q^{\pi}(s,a) &\overset{\text{def}}{=} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t}\,\middle|\,S_0=s,A_0=a\right] \\
&\overset{(1)}{=} \mathbb{E}_{\pi}\left[R_0+\gamma\sum_{t=0}^{\infty}\gamma^{t}R_{t+1}\,\middle|\,S_0=s,A_0=a\right] \\
&\overset{(2)}{=} r(s,a)+\gamma\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t+1}\,\middle|\,S_0=s,A_0=a\right]
\end{aligned}
\tag{19}
$$

式中第 (1) 步拆出第一项 R_0 和系数 γ 并调整索引（$t\leftarrow t+1$），第 (2) 步代入 $R_0=r(S_0,A_0)$ 并将其与系数 γ 从期望中提出来。表达式应用全概率公式可得：

$$
r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t+1}\,\middle|\,S_0=s,A_0=a,S_1=s'\right] \tag{20}
$$

由马尔可夫性，给定 $S_1=s'$ 条件的情况下未来与 (s_0,a_0) 无关，故消去相应条件得：

$$
Q^{\pi}(s,a)=r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t+1}\,\middle|\,S_1=s'\right] \tag{21}
$$

再利用时移性质 Equation 14 将期望替换为 $V^{\pi}(s')$ 即可得 Equation 13。

1.3. Bellman Expectation Equations

我们可以推导出关于值函数的递推表达式：

$$
\begin{aligned}
V^{\pi}(s) &= \mathbb{E}_{A\sim\pi(\cdot|s),S'\sim P(\cdot|s,A)}[r(s,A)+\gamma V^{\pi}(S')] \\
&= \sum_{a}\pi(a\mid s)\sum_{s'}P(s'\mid s,a)[r(s,a)+\gamma V^{\pi}(s')]
\end{aligned}
\tag{22}
$$

该递推式也被称为**贝尔曼期望公式**（Bellman expectation equation），证明见后。动作值函数的版本是：

$$
\begin{aligned}
Q^{\pi}(s,a) &= r(s,a)+\gamma\mathbb{E}_{S'\sim P(\cdot|s,a),A'\sim\pi(\cdot|S')}[Q^{\pi}(S',A')] \\
&= r(s,a)+\gamma\sum_{s'}P(s'\mid s,a)\sum_{a'}\pi(a'\mid s')Q^{\pi}(s',a')
\end{aligned}
\tag{23}
$$

关于期望与条件期望的符号约定：

在课件中，值函数是这样定义和展开的：

$$V^\pi(s) := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t r_t \;\middle|\; s_0 = s\right] = \mathbb{E}\Big[r(s,a) + \gamma V^\pi(s') \;\Big|\; \substack{a\sim\pi(\cdot|s)\\ s'\sim P(\cdot|s,a)}\Big] \tag{24}$$

其记号存在诸多混淆之处，不过确实如果处处都写完整比较繁琐，表意到位即可。后文中我们尽量不采用混淆的写法，但同时也在此澄清课件写法的含义。

首先，形如 $s_0 = s$ 等写法混淆了随机变量和样本，按照我们之前的定义应该写 $S_0 = s$ 用以区分。

第二，求期望需要明确的是随机变量及其采样自什么分布。值函数定义中条件期望的条件约束了初始状态 s_0，求和式中则使用了各时刻的奖励 r_t，比起列举所有变量，我们可以直接令这里的随机变量是服从轨迹概率分布的随机序列 T，即前述 $\mathrm{T} \sim p_{\mathrm{T}}(\cdot; \mathcal{M}, \pi)$，其包含了所有的状态、动作和奖励，而 $\mathbb{E}_\pi[\cdot]$ 的含义如 Equation 6 所述。

第三，一般不会像第二个等号这样将随机变量和及其分布写到条件期望条件的位置上去，而是写在 $\mathbb{E}$ 下标处或者直接省略。

如此我们就统一了 Equation 22 和 Equation 24 的形式。接下来补充部分其他公式在课件里的写法，方便对照：

$$J^\pi := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t r_t\right] = \mathbb{E}_\rho[V^\pi(s_0)] = \mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t r(s_t,a_t) \;\middle|\; \substack{s_0\sim\rho(\cdot),\, a_t\sim\pi(\cdot|s_t)\\ s_{t+1}\sim P(\cdot|s_t,a_t)}\right] \tag{25}$$

$$Q^\pi(s,a) := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty}\gamma^t r_t \;\middle|\; \substack{s_0=s\\ a_0=a}\right] = r(s,a) + \gamma\mathbb{E}\Big[Q^\pi(s',a') \;\Big|\; \substack{s'\sim P(\cdot|s,a)\\ a'\sim\pi(\cdot|s')}\Big] \tag{26}$$

此外还有需要特别说明的一点，关于期望的嵌套（条件期望的 tower property 中经常出现）。例如对随机变量 X 和 Y 的表达式 $f(X,Y)$ 求期望，则有：

$$\mathbb{E}_{X,Y}[f(X,Y)] = \mathbb{E}_{X,Y}\big[\mathbb{E}_{X,Y}[f(X,Y) \mid Y = Y]\big] \tag{27}$$

这里条件 $Y = Y$ 的写法是令人迷惑的。实际上，左侧的 Y 是内侧 $\mathbb{E}$ 的随机变量，而右侧的 Y 是外侧期望的随机变量（对内侧期望来说是常数），所以为了不混淆最好重命名一下内侧的随机变量：

$$\mathbb{E}_{X,Y}[f(X,Y)] = \mathbb{E}_{X,Y}\big[\mathbb{E}_{X',Y'}[f(X',Y') \mid Y' = Y]\big] \tag{28}$$

不过后文例如嵌套 $\mathbb{E}_\pi$ 时不高兴一个一个重命名并说明，有时就不重命名，直接简写一个 Y 表示这个条件：

$$\mathbb{E}_{X,Y}[f(X,Y)] = \mathbb{E}_{X,Y}\big[\mathbb{E}_{X,Y}[f(X,Y) \mid Y]\big] \tag{29}$$

此后关于这类符号问题不再赘述。

关于贝尔曼期望公式的证明：

我们先证明值函数的版本：

$$
\begin{aligned}
V^{\pi}(s) &\overset{\text{def}}{=} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t} \;\middle|\; S_0=s\right] \\
&\overset{(1)}{=} \mathbb{E}_{\pi}\left[R_0+\gamma\sum_{t=0}^{\infty}\gamma^{t}R_{t+1} \;\middle|\; S_0=s\right] \\
&\overset{(2)}{=} \mathbb{E}_{\pi}\left[R_0+\gamma\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t+1} \;\middle|\; S_0=s,A_0,S_1\right] \;\middle|\; S_0=s\right] \\
&\overset{(3)}{=} \mathbb{E}_{\pi}\left[R_0+\gamma\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R_{t+1} \;\middle|\; S_1\right] \;\middle|\; S_0=s\right] \\
&\overset{(4)}{=} \mathbb{E}_{\pi}[R_0+\gamma V^{\pi}(S_1)\mid S_0=s] \\
&\overset{(5)}{=} \mathbb{E}_{A_0\sim\pi(\cdot|s),S_1\sim P(\cdot|s,A_0)}[r(s,A_0)+\gamma V^{\pi}(S_1)] \\
&\overset{(6)}{=} \mathbb{E}_{A\sim\pi(\cdot|s),S'\sim P(\cdot|s,A)}[r(s,A)+\gamma V^{\pi}(S')]
\end{aligned}
\tag{30}
$$

分别阐释一下上式每个步骤的理由：最开始的 def 式是定义，接下来第 (1) 步从求和中拆出第一项并调整了一下索引（$t\leftarrow t+1$）。

第 (2) 步是应用迭代期望公式（tower property of conditional expectation）将里面套上条件期望，引入额外条件 A_0 和 S_1。这里两个变量来自外层 $\mathbb{E}_{\pi}$，对内层可视为已知量，和内层 $\mathbb{E}_{\pi}$ 的变量名重复了所以直接写符号而不是 $A_0=A_0$ 等。

第 (3) 步内层 $\mathbb{E}_{\pi}$ 利用马尔可夫性，已知 $t=1$ 时状态为 S_1，则条件分布和之前时刻的 $S_0=s$、A_0 都无关，从而省去条件。第 (4) 步直接利用 Equation 14 中的时移性质。

第 (5) 步是将轨迹分布 $p_{\mathrm{T}}^{\pi}(\cdot)$ 对 (A_0,S_1) 的边缘分布取出来。具体地，轨迹分布是链上所有随机变量的联合分布，而第 (4) 步所求期望仅依赖 S_0、A_0、R_0 和 S_1，故无需考虑之后的 A_1 等；余下的 $S_0=s$ 和 $R_0=r(S_0,A_1)$ 由条件和定义消去，只留下对 (A_0,S_1) 的边缘分布。最后第 (6) 步只是符号重命名（$S_1\to S'$，$A_0\to A$），得到更通用的形式。

这里的证明思路和Equation 13 的证明思路基本一致，大致都是引入下一时刻的随机变量以形成递推的结构。

其实不止是思路一致了，基于之前的证明完全可以直接将 Equation 13 代入 Equation 12 得到 Equation 22、将 Equation 12 代入 Equation 13 得到 Equation 23，这里就当提供一个一体的版本。

1.4. Optimal Bellman Equations

到现在为止，我们定义的值函数和动作值函数都是基于策略 π 的，通过调整策略可以得到不同的值函数集合。

我们定义**最优值函数**（optimal value function）和**最优动作值函数**（optimal action-value function）：

$$V^*(s) \coloneqq \max_\pi V^\pi(s) \tag{31}$$

$$Q^*(s,a) \coloneqq \max_\pi Q^\pi(s,a) \tag{32}$$

注意该定义并不能保证存在一种最优策略使得每个状态 s 或状态动作对 (s,a) 都能取到这个最优值。不过我们就假设存在最优策略 π^*：

$$\exists \pi^*, \quad V^{\pi^*}(s) \equiv V^*(s) \geq V^\pi(s), \quad \forall \pi, \forall s \in \mathcal{S} \tag{33}$$

动作值函数同理。实际上这个假设对大部分情况都成立，因为我们用 $\pi(a \mid s)$ 对策略建模，本身就可以针对每个 s 设计策略，而将所有状态下的最优子策略拼在一起即可得到最优策略。

之前推导了值函数和动作值函数的关系，这里也有最优值函数和最优动作值函数的关系：

$$V^*(s) = \max_a Q^*(s,a) \tag{34}$$

关于最优值函数和最优动作值函数关系的证明：

首先由定义和 Equation 12 有：

$$V^*(s) = \max_\pi V^\pi(s) = \max_\pi \sum_a \pi(a \mid s) Q^\pi(s,a) \tag{35}$$

接下来我们先证明其 $\leq \max_a Q^*(s,a)$。由定义，对任意 π 有：

$$\sum_a \pi(a \mid s) Q^\pi(s,a) \leq \sum_a \pi(a \mid s) \max_{a'} Q^*(s,a') = \max_a Q^*(s,a) \tag{36}$$

然后证明其 $\geq \max_a Q^*(s,a)$。构造一个策略 $\tilde{\pi}$ 如下：在状态 s 采取动作 $\tilde{a} = \arg\max_a Q^*(s,a)$，即有 $Q^*(s,\tilde{a}) = \max_a Q^*(s,a)$，之后按照最优策略执行。该策略在状态 s 的期望回报为：

$$V^{\tilde{\pi}}(s) = \sum_a \tilde{\pi}(a \mid s) Q^{\tilde{\pi}}(s,a) = Q^*(s,\tilde{a}) = \max_a Q^*(s,a) \tag{37}$$

又由于 $V^*(s)$ 是所有策略中可以取到的最大期望回报，故有：

$$\max_\pi \sum_a \pi(a \mid s) Q^\pi(s,a) = V^*(s) \geq V^{\tilde{\pi}}(s) = \max_a Q^*(s,a) \tag{38}$$

两个不等式联合得到 Equation 34。

最优值函数和最优动作值函数也有迭代形式，即**最优贝尔曼方程**（optimal Bellman equations），证明见后：

$$\begin{aligned} V^*(s) &= \max_a \Big[r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot \mid s,a)} [V^*(S')] \Big] \\ &= \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right] \end{aligned} \tag{39}$$

$$\begin{aligned} Q^*(s,a) &= r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot \mid s,a)} \left[\max_{a'} Q^*(S',a') \right] \\ &= r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q^*(s',a') \end{aligned} \tag{40}$$

或者写成积分形式，意思都一样，只是连续和离散的区别：

$$Q^*(s,a) = r(s,a) + \gamma \int P(s' \mid s,a) \max_{a'} Q^*(s',a') \, \mathrm{d}s' \tag{41}$$

对应的最优策略为在任意状态下都选取令最有动作值函数最大的动作，即：

$$\pi^*(a \mid s) = 1, \quad \text{iff } a = \arg\max_{a'} Q^*(s,a') \tag{42}$$

关于最优贝尔曼方程的证明：

从定义出发，利用前面证明的一些结论很容易得到最优动作值函数的版本：

$$\begin{aligned} Q^*(s,a) &= \max_\pi Q^\pi(s,a) \\ &= \max_\pi \Big[r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot \mid s,a)} [V^\pi(S')] \Big] \\ &\overset{*}{=} r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot \mid s,a)} \left[\max_\pi V^\pi(S') \right] \\ &= r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot \mid s,a)} [V^*(S')] \\ &= r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot \mid s,a)} \left[\max_{a'} Q^*(S',a') \right] \end{aligned} \tag{43}$$

其中需要注意的是 $*$ 式，这一步将 $\max_\pi$ 放进了期望中，不具备一般性，因为前者要求同一个策略在所有可能后续状态 S' 上表现最佳，放进期望后则允许对每个状态 S' 分别选取最佳策略。故欲使该步成立，需要满足前述最优策略存在的假设，即 Equation 33。

接下来代入 Equation 34 即可直接得到最优值函数的版本：

$$
\begin{aligned}
V^*(s) &= \max_a Q^*(s,a) \\
&= \max_a \Big[r(s,a) + \gamma \mathbb{E}_{S' \sim P(\cdot|s,a)} [V^*(S')] \Big]
\end{aligned}
\tag{44}
$$

考虑最优策略价值函数 $J^* := \max_\pi J^\pi$：

$$
\begin{aligned}
\max_\pi J^\pi &= \max_\pi \mathbb{E}_\rho [V^\pi(S_0)] \\
&\overset{*}{=} \mathbb{E}_\rho \Big[\max_\pi V^\pi(S_0) \Big] = \mathbb{E}_\rho [V^*(S_0)] \\
&= \mathbb{E}_\rho \Big[\max_a Q^*(S_0, a) \Big] \\
&\overset{*}{=} \mathbb{E}_{S_0 \sim \rho(\cdot)} \Big[\max_a \Big(r(S_0,a) + \gamma \mathbb{E}_{S' \sim P(\cdot|S_0,a)} \Big[\max_{a'} Q^*(S', a') \Big] \Big) \Big] \\
&= \mathbb{E}_{S \sim \rho(\cdot)} \Big[\max_a \Big(r(S,a) + \gamma \mathbb{E}_{S' \sim P(\cdot|S,a)} \Big[\max_{a'} \max_\pi Q^\pi(S', a') \Big] \Big) \Big]
\end{aligned}
\tag{45}
$$

其中两个 $*$ 式的含义仍表示需要 Equation 33 的最优策略存在；前者是为了将 $\max_\pi$ 放进期望中，后者是证明最优动作值函数贝尔曼方程时用到了该假设；最后一步依旧符号重命名。

2. Value-Based Learning

2.1. Policy Iteration and Value Iteration

（略）以迭代的方式求解最优 $V^*(s)$，在其基础上贪心制定最优策略。

2.2. Tabular Q-Learning and SARSA

既然可以迭代求解 $V^*(s)$，索性直接求 $Q^*(s,a)$，更方便求解策略 $a^* = \arg\max_a Q^*(s,a)$。考虑状态和动作空间离散的情况，$Q(s,a)$ 实际上是表格式的（tabular），包含共 $|\mathcal{S}| \times |\mathcal{A}|$ 个元素。

过程类似值迭代，先初始化：

$$Q(s,a) \leftarrow 1, \quad \forall s,a \tag{46}$$

让智能体在环境中进行探索，用**时序差分**（temporal difference，TD）目标进行更新。具体地，任意 t 时刻发生状态转移时得到一组新数据 (s_t, a_t, r_t, s_{t+1})，定义目标：

$$y_t := r_t + \gamma \max_a Q(s_{t+1}, a) \tag{47}$$

此处的**目标**（target）可视作对真实最优动作值函数 $Q^*(s_t,a_t)$ 的估计（节末注释中会更详细地讨论），用它搭配一个学习率 α 对当前 $Q(s_t,a_t)$ 进行更新：

$$Q(s_t,a_t) \leftarrow (1-\alpha)Q(s_t,a_t) + \alpha y_t \tag{48}$$

这种形式形似“插值”，或者另一种更像“更新”的等价形式也比较常用：

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha(y_t - Q(s_t,a_t)) \tag{49}$$

其中 $y_t - Q(s_t,a_t)$ 被称为**时序差分误差**（TD error）。

值得注意的是，智能体探索时所实际执行的策略即**行为策略**（behavior policy）是基于 Q 估计值计算的，而更新式中 $\max_a Q(s_{t+1},a)$ 说明更新时所用的策略即**目标策略**（target policy）是取当前估计下的最优策略。故在 Q-Learning 中，行为策略和目标策略不一致，这种类型的算法称为 off-policy 算法。关于 on-/off-policy 会在后文 Section 2.4.1 中更详细地讨论。

与 Q-Learning 相对应，自然也有 on-policy 的 SARSA 算法，其每次状态转移采样得到 $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$（这五项的首字母也是算法得名原因），选取的目标为：

$$y_t^{\text{SARSA}} := r_t + \gamma Q(s_{t+1}, a_{t+1}) \tag{50}$$

该目标直接计算自行为策略采样的样本，用于目标策略的更新，二者一致，故为 on-policy 的。

表格式 Q-Learning 最大的问题在于**维度诅咒**（curse of dimensionality），状态和动作空间较大时 Q 的元素数量指数级上升，让学习几乎成为不可能，也难以处理状态或动作连续的情况。

关于一个有效目标的设计及 Q-Learning 和 SARSA 的收敛性：

我们需要澄清一个重要的原则，关于所谓的目标（target）应该怎样去选取，以回答问题：为什么前面 SARSA、Q-Learning 等算法采用现在这样的目标？它们为什么有效？

一个重要的判断是：一个有效目标的条件期望必须等同于我们要学习的对象。注意这不是一个规则，而是在迭代更新方法框架下得到的一个方便的推论。

我们得举点例子说明，例如在 Q-Learning 中我们希望学习的是最优动作值函数 $Q^*(s,a)$，定义随机变量 $Y_t^* := r(S_t, A_t) + \gamma \max_a Q^*(S_{t+1}, a)$，由最优贝尔曼方程 Equation 40 可知其条件期望满足：

$$\mathbb{E}_\pi[Y_t^* \mid S_t = s, A_t = a] = Q^*(s,a) \tag{51}$$

即我们要学习的目标。这里 $\mathbb{E}_\pi$ 的 π 可以不写，因为 (s,a) 已经给定，这个期望和策略无关，这里只是强调一下，如果是 V 就无法省略了。于是由 Y_t^* 我们可以设计目标为其样本：

$$y_t^* := r(s_t, a_t) + \gamma \max_a Q^*(s_{t+1}, a) \tag{52}$$

然而，这个目标虽然有效，却无法实际应用，因为我们根本不知道 $Q^*(s_{t+1}, a)$ 的真值，Q^* 本来就是我们要求的东西。这实际上已经是另一个问题了，解决方法是用当前参数估计 $Q(s_{t+1}, a)$ 来近似它，于是就得到了最终 Equation 47 中的 y_t。这么做的依据是通过最优贝尔曼算子可以证明 Q 将随迭代收敛到不动点 Q^*，所以 Q-Learning 目标的合理性其实是通过两重依据来保障的：

$$\text{objective } Q^* \overset{!}{=} \mathbb{E}[\text{target } Y_t^*] \leftarrow \mathbb{E}[\text{approximated target } Y_t] \tag{53}$$

这里等号上加叹号表示我们要求其成立。

我们再看 SARSA 的例子，定义 $Y_t^{\text{SARSA}} := r(S_t, A_t) + \gamma Q^\pi(S_{t+1}, A_{t+1})$，由贝尔曼期望公式 Equation 23，其条件期望满足：

$$\mathbb{E}_\pi\left[Y_t^{\text{SARSA}} \mid S_t = s, A_t = a\right] = Q^\pi(s,a) \tag{54}$$

其目标样本即 Equation 50 中的形式。但这样我们发现一个问题，SARSA 好像学习的是 Q^π 而非最优 Q^*，也就是说如果行为策略 π 是固定的，那么 SARSA 实际上做的就是策略评估（policy evaluation）的工作。但是没关系，我们会采用例如 ϵ-贪婪等探索策略，只要有概率以更贪心的策略去采样，就相当于是进行了策略优化（policy improvement），二者相结合就得到和 Section 2.1 中一样的“策略评估 + 策略优化”架构，最终将会收敛到最优策略 π^* 和 Q^*。所以 SARSA 的收敛性和最优性其实也是通过两重依据来保障的：

$$\text{optimal } Q^* \leftarrow \text{objective } Q^\pi \overset{!}{=} \mathbb{E}\left[\text{target } Y_t^{\text{SARSA}}\right] \tag{55}$$

之后我们基于相同结构的目标构造损失函数使用残差梯度法更新时，会发现结果是有偏的。正儿八经证明上述 Q-Learning 和 SARSA 的收敛性和无偏性还是要通过最优贝尔曼方程或贝尔曼期望方程的去证明不动点的存在性和唯一性。

2.3. Value Approximation

为了解决表格式算法 cannot scale 的问题，我们可以将 V 或 Q 函数参数化，用相对少的参数去表达函数值：

$$v_\theta(s) \approx V^\pi(s), \quad q_\theta(s,a) \approx Q^\pi(s,a) \tag{56}$$

其中 θ 表示参数。参数化的方式有很多，例如 SoB（sum of basis）、神经网络等。

参数化后的值函数自然无法再用表格式算法的方式进行更新，通常我们采用梯度更新的方式对参数进行更新。于是我们现在需要考虑的问题变为：1、用什么**损失函数**（loss function）；2、用什么计算**梯度**（gradients）。

2.3.1. Dataset

先规范一些符号问题，训练要有数据集，而数据集可以用多种方式表达。我们最关注的往往是状态转移，每次状态转移都会带来一条关于“在某状态采取某行动会产生什么后果”的信息，所以在部分算法中我们以 $(s_k, a_k, r_k, s_{k+1}, a_{k+1})$ 的元组作为最小单位进行存储，而不在意其采样自何种策略、时间戳是多少。比如普通的 Q-Learning 也不在乎 a_{k+1}，可以只存：

$$\mathcal{D} = \{(s_k, a_k, r_k, s_{k+1})\}_{k=1}^n \tag{57}$$

实际实现中通常还会存下一个状态 s_{k+1} 是否是终端状态方便从目标中剔除后续项，例如直接加一个 $\text{done}_k \in \{0,1\}$：

$$\mathcal{D} = \{(s_k, a_k, r_k, s_{k+1}, \text{done}_k)\}_{k=1}^n \tag{58}$$

但这也不绝对，比如说很多 on-policy 算法需要知道样本对应的采样策略，例如之后的 PPO 算法会用到样本对应的采样概率（的对数值）和值函数估计：

$$\mathcal{D} = \{(s_k, a_k, r_k, s_{k+1}, \text{done}_k, \log \pi_{\text{old}}(a_k \mid s_k), V^{\text{old}}(s_k))\}_{k=1}^n \tag{59}$$

而且毕竟是 on-policy 算法，也没什么写成集合 $\mathcal{D}$ 的必要，按顺序 s_{k+1} 一般就是下一条数据的 s_k，可以不存它。还有一些基于 Monte-Carlo 的算法需要整条轨迹信息，需要保持采样轨迹的完整结构：

$$\mathcal{D} = \{\tau_k\}_{k=1}^n, \quad \tau_k = \left(s_0, a_0, r_0, s_1, ..., r_{n_k-1}, s_{n_k}\right) \tag{60}$$

举例这么多就是说明这需要根据实际需求选择。现在我们的需求是为下面提出的方法铺垫，故作一些简化假设。假设所有样本都采集自固定的策略 π，并且无所谓顺序，以状态转移为最小单元存储：

$$\mathcal{D}_\pi = \left\{(s_k, a_k, r_k, s_{k+1}, a_{k+1})\right\}_{k=1}^{n} \tag{61}$$

为了方便证明，我们用随机变量刻画采样过程：

$$D = (S_t, A_t, R_t, S_{t+1}, A_{t+1}) \sim P_{\mathcal{D}_\pi}(\cdot) \tag{62}$$

$P_{\mathcal{D}_\pi}(\cdot)$ 是采样分布，只是为了完善记号，之后我们就用 $\mathbb{E}_{\mathcal{D}_\pi}[\cdot]$ 代表 $\mathbb{E}_{D\sim P_{\mathcal{D}_\pi}(\cdot)}[\cdot]$。

2.3.2. Residual Value Gradients

最常用的损失函数仍然是**均方误差**（mean-squared error，MSE），考虑值函数估计 $v_\theta(s)$ 和目标之间在数据集 $\mathcal{D}_\pi$ 上的均方误差：

$$\mathcal{L}[\theta] \coloneqq \mathbb{E}_{\mathcal{D}_\pi}\left[\left(\overbrace{\underbrace{R_t + \gamma v_\theta(S_{t+1})}_{\text{target } Y_t} - v_\theta(S_t)}^{\text{TD-error}}\right)^2\right] \tag{63}$$

动作值函数的版本如下：

$$\mathcal{L}[\theta] \coloneqq \mathbb{E}_{\mathcal{D}_\pi}\left[\left(\overbrace{\underbrace{R_t + \gamma q_\theta(S_{t+1}, A_{t+1})}_{\text{target } Y_t} - q_\theta(S_t, A_t)}^{\text{TD-error}}\right)^2\right] \tag{64}$$

可以看出它们的设计是对应 on-policy 策略的，而后者实际上就是 SARSA。直接采用梯度下降法优化损失函数，这一套损失函数设计与梯度更新方式称为**残差梯度**（residual gradients）法，平方项内的部分可视为**残差**（residues）。

残差 $Y_t - q_\theta(S_t, A_t)$ 的形式和 Equation 49 中 $y_t - Q(s_t, a_t)$ 也是一致的，就如前文所述，Y_t 是条件期望等于欲学习对象的目标，而残差本质上就是在衡量当前估计与目标的距离，优化该损失函数就是在尝试推动 q_θ 收敛到欲学习的对象。

但要注意，目标 Y_t 中用到了 $q_\theta(S_{t+1}, A_{t+1})$，它也是基于当前参数 θ 的一个近似值而不是真实的 $Q^*(S_{t+1}, A_{t+1})$，这导致它实际无法真的令 q_θ 收敛到 Q^*。类似这种“用估计的参数当目标去优化参数估计”的行为我们称为**自举**（bootstrapping），也是引发问题的根源。细节上的不同使得这种自举迭代可能让结果收敛（例如值迭代和表格式 Q-Learning），也可能导致发散或产生稳态误差。

在 Q-Learning 的迭代更新框架下可以证明其最终仍然收敛到最优值，但此处残差梯度法的结果略有不同，接下来就具体讨论这个问题。为了更清晰地展示参数的变化，我们以表格式 Q-Learning 为例进行说明。它也可以视作一种参数化，只不过参数就是所有表格值 $\theta \coloneqq \boldsymbol{Q} = \left\{Q_{s,a}\right\} \in \mathbb{R}^{|\mathcal{S}|\times|\mathcal{A}|}$。写出 Q-Learning 观察到转移 (s_t, a_t, r_t, s_{t+1}) 后的一步损失：

$$\mathcal{L}[\boldsymbol{Q}] \coloneqq \left(r_t + \gamma Q_{s_{t+1},a^*} - Q_{s_t,a_t}\right)^2, \quad a^* \coloneqq \arg\max_a Q_{s_{t+1},a} \tag{65}$$

及一步梯度下降更新式：

$$\boldsymbol{Q} \leftarrow \boldsymbol{Q} - \frac{\alpha}{2}\nabla_{\boldsymbol{Q}}\mathcal{L}[\boldsymbol{Q}] \tag{66}$$

由于参数就是表格元素，各元素对 $\boldsymbol{Q}$ 求导就是对应位置的单位冲激函数：

$$\nabla_Q Q_{s,a} = \delta_{s,a}, \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A} \tag{67}$$

其中 $\delta_{s,a} = 1$ 当且仅当自变量为 (s,a)，课件则采用的类似 $\delta(a=\cdot, s=\cdot)$ 的写法。于是表格每项的更新为：

$$Q_{s,a} \leftarrow Q_{s,a} - \alpha\left(r_t + \gamma Q_{s_{t+1},a^*} - Q_{s_t,a_t}\right)\left(\gamma\delta_{s_{t+1},a^*} - \delta_{s_t,a_t}\right), \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A} \tag{68}$$

由于其中两个 δ 函数的存在，这里的一步更新实际上只会更新两个元素：

$$\begin{aligned} Q_{s_t,a_t} &\leftarrow (1-\alpha)Q_{s_t,a_t} + \alpha\left(r_t + \gamma Q_{s_{t+1},a^*}\right) \\ Q_{s_{t+1},a^*} &\leftarrow (1-\alpha\gamma^2)Q_{s_{t+1},a^*} - \alpha\gamma\left(r_t - Q_{s_t,a_t}\right) \end{aligned} \tag{69}$$

第一部分 Q_{s_t,a_t} 的更新没有什么问题，$r_t + \gamma Q_{s_{t+1},a^*}$ 就是 y_t，和之前 Equation 48 中完全一致；但这里还多出了第二部分 Q_{s_{t+1},a^*} 的更新，这就是与前文标准的表格式 Q-Learning 不一致的地方了。

这会导致两个问题，首先最重要的是多了这一项的残差梯度是**有偏的**（biased），无法再令参数收敛到正确值，证明见后。我们可以先从现有结论上接受这一点：利用贝尔曼最优算子可以证明 Q^* 是最优贝尔曼方程的唯一不动点解，且标准表格式 Q-Learning 通过迭代更新会收敛到这个解；但如前所述，现在残差梯度化简后比迭代更新的式子多了一条，多半结果也会有偏差。

第二是违背了**因果性**（causality）。具体地，更新的本质是树立一个目标对当前参数 Q_{s_t,a_t} 进行调整，但残差梯度同时还去更新了未来状态的参数 Q_{s_{t+1},a^*}。这种参数估计和目标同时在变的情况自然会导致更新过程不稳定，参数信息传播和学习速度变慢。

关于基于一步 TD 目标的残差梯度法在固定策略数据上优化结果有偏的证明：

标题叠了一堆甲，主要是关于几点假设：1、用残差梯度法直接优化；2、以值函数采样目标 $y_t := r_t + \gamma v_\theta(s_{t+1})$ 为例；3、数据集采样自固定策略 π。

如果按之前讨论的目标设计要求，该目标的条件期望正是我们希望估计的 $V^\pi(s)$，没有问题；但我们从迭代更新框架迁移到损失函数优化问题上这一过程只能说是直觉的，实际上没道理认为目标设计一致就能直接用，仍然需要证明，而结果也确有问题。

接下来我们具体证明直接对 Equation 63 应用残差梯度更新所得结果是有偏的。我们不是去证明迭代结果偏离，而是证明损失函数有偏（源于目标中采用的是对值函数的估计而非真实值函数），从而说明优化结果有偏。

TODO Assignment A1.3 (70)

动作值函数的目标道理也差不多，不再赘述。

2.3.3. Value Semi-gradients

为了解决前述问题，标准 Q-Learning 常采用的是**半梯度**（semi-gradients）更新的方式。比如上节例子中残差梯度法同时更新 Q_{s_t,a_t} 和 Q_{s_{t+1},a^*}，半梯度法则只用 Q_{s_t,a_t} 这一部分梯度更新，从而和表格式 Q-Learning 保持一致并得以套用其收敛性结论。

回到参数化损失 Equation 63，问题本质是目标 y_t 依赖当前参数估计 θ，那么我们可以通过将目标中的 v_θ 替换为另一组迟滞参数驱动的 $v_{\theta'}$ 来削减这种耦合：

$$\mathcal{L}[\theta] := \mathbb{E}_{\mathcal{D}_\pi}\left[\left(\underbrace{R_t + \gamma v_{\theta'}(S_{t+1})}_{\text{bootstrapped target } Y_t} - v_\theta(S_t)\right)^2\right] \tag{71}$$

这样在更新时就有 $\nabla_\theta v_{\theta'}(S_{t+1}) = 0$，对应舍去的那一部分梯度。实际更新时 θ' 的数值是和 θ 相等的（$\theta' = \theta$），所以说明白点就只是反向传播时切断了 θ' 的这一部分梯度，前向传播时照旧。

以上更新方式称为**半梯度时序差分学习**（semi-gradient TD-learning）。在 PyTorch 具体实现中，给相应项加上 `.detach()` 即可实现只截断某一部分反向传播的操作。该方案比残差梯度法更快且无偏，对线性模型可以证明收敛，例如前文表格 Q-Learning 的例子。

但采用神经网络等模型参数化值函数时就很容易发散，这是由于模型估计误差、自举性等因素复杂综合导致的。引入 θ' 只能弱化耦合而无法消除，因为终究无法得到真实的 Q^* 充当优化目标。

若要更进一步，可以考虑**神经拟合 Q 迭代**（neural-fitted Q-iteration，NFQ）算法，这是第一个成功的深度强化学习算法，就是迭代比较慢。相比普通的半梯度学习，它会在每次阶段性收敛后再将 θ 同步到 θ'，一步一个脚印显著降低发散的概率。

2.3.4. Online Q-Learning

接下来讨论一个实例的具体实现细节，考虑用半梯度 TD 学习实现在线 Q-Learning，神经网络作为参数化模型，“在线”表示从实时探索中采样所得样本中学习。这是最基础的深度 Q-Learning，为后文 DQN 做铺垫。

首先有关参数化模型 $q_\theta(s,a)$，自然的设计是将 s 和 a 作为输入层，输出层输出 Q 值。但对于离散动作 Q-Learning，更新时需要反复计算 $\max_a q_\theta(s_{t+1},a)$，如果能一次前向传播计算出所有动作 a 对应的 $q_\theta(s_{t+1},a)$ 值（输出一个向量）相比依次计算最后取最值自然是更高效的。所以我们将状态作为输入层，输出层则让每个动作对应一个输出（one output/head per action）。

引入半梯度法后，模型可以表达为：

$$\underbrace{q_\theta(s_t,a_t)}_{\text{value}} \overset{!}{=} \max_\pi \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k} \,\middle|\, s_t,a_t\right] \approx \underbrace{r(s_t,a_t) + \gamma\mathbb{E}\left[\max_{a'} q_{\theta'}(S_{t+1},a')\right]}_{\text{target}} \tag{72}$$

用 PyTorch 实现节选如下：

```python
from torch.optim import RMSprop
from torch.nn.functional import mse_loss
```

```
3
4  optimizer = RMSprop(q.parameters())
5
6  while True:
7      # sample episode from environment
8      state, action, reward, term, next_state = environment.sample(q)
9
10     # compute left and right side of Bellman eq.
11     value = q(state).gather(dim = -1, index = action)
12     target = reward + gamma * (~term * q(next_state).max(dim = -1)[0])
13
14     # gradient descent step on supervised regression loss
15     optimizer.zero_grad()
16     mse_loss(value, target.detach()).backward()
17     optimizer.step()
```

其中，可以观察到半梯度法中 `target.detach()` 的使用。此外，代码中的 `term` 指示是否是终止状态（terminal state），确保在终止状态不再考虑未来的奖励。

2.4. Stabilization Issues and Techniques

前文讨论过，朴素的半梯度在线 Q-Learning 很容易发散。从机器学习的角度来看，这种不稳定性源于其违背了一系列假设，例如：

1. 回归目标不平稳（non-stationary），即前述的自举问题；
2. 探索策略会改变训练数据分布，采样数据也随之改变；
3. 网络随着训练会遗忘旧样本，即灾难性遗忘（catastrophic forgetting）；
4. 状态转移之间强相关，不满足**独立同分布**（independent and identically distributed，i.i.d.）假设。

接下来我们详细讨论其中较关键的一些问题和它们的解决方案。

2.4.1. Experience Replay

在强化学习中没有传统监督学习的训练集和测试集划分环节，所有数据来自与环境交互的采样。采样和更新采用的策略可能一致可能不一致，即前文提过的 on-policy 和 off-policy 采样。

一般我们将希望评估或者优化的策略记为目标策略 π，将探索中实际采用产生样本的策略记为行为策略 b。在我们优化目标策略 π 时，若更新所用的样本也是策略 $b = \pi$ 采集的，那么算法就是 on-policy 的，它相对稳定且若学习 V 只需要考虑状态输入，维度较低，但若策略变化就需要重新采样数据；若所用的样本是某个其他的行为策略 $b \neq \pi$ 采集的，那么算法就是 off-policy 的，若学习 Q 它需要更大的输入空间（状态和动作），但可以复用旧的或者其他智能体的经验。

> **V/Q 的选择与 on-/off-policy 的关系：**
>
> 学习 V 还是 Q 这件事和 on-/off-policy 本没有直接的联系，但 on-policy 学 V 和 off-policy 学 Q 是比较自然和直接的。

实际上，区分 on-/off-policy 的根本依据不是看行为策略和目标策略是否一致，因为理论上 off-policy 的算法都可以“退化”到 on-policy，把实时数据当历史数据用就可以了；但 on-policy 通常不能当 off-policy 算法用，所以我们需要看的应该是一个算法是否有能力利用不同于目标策略的行为策略产生的样本来学习优化目标策略。

而是否能利用其他策略提供的样本，主要就看代入这个样本之后目标 y_t 是否还合法、有效（关于合理的样本设计，回顾 Section 2.2 末注释中的讨论）。

如果要学习 $V^\pi(s)$，应设计如 Equation 63 中的目标 $y_t^\pi := r_t + \gamma v_\theta(s_{t+1})$。由贝尔曼期望公式，对应的随机变量 $Y_t^\pi := R_t + \gamma v_\theta(S_{t+1})$ 的条件期望满足：

$$\begin{aligned}\mathbb{E}_\pi[Y_t^\pi \mid S_t = s] &= \mathbb{E}_\pi[R_t + \gamma v_\theta(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)[r(s,a) + \gamma V^\pi(s')] \\ &= V^\pi(s)\end{aligned} \tag{73}$$

其中，有 $A_t \sim \pi(\cdot \mid S_t = s)$ 和 $S_{t+1} \sim P(\cdot \mid S_t = s, A_t)$。如果代入由某个行为策略 $b \neq \pi$ 采集的样本，会导致得到的 Y_t^b 的条件期望中 $A_t \sim b(\cdot \mid s)$，而：

$$\begin{aligned}\mathbb{E}_\pi[Y_t^b \mid S_t = s] &= \sum_a b(a \mid s) \sum_{s'} P(s' \mid s, a)[r(s,a) + \gamma V^\pi(s')] \\ &\neq V^\pi(s)\end{aligned} \tag{74}$$

故 y_t^b 不再是一个能正确学习 $V^\pi(s)$ 的目标，所以这种直接 TD 估计 $V^\pi(s)$ 的更新式通常只能是 on-policy 的。

相同的思路下，如果要学习 $Q^\pi(s,a)$，计算目标的条件期望时直接给定了条件 $S_t = s, A_t = a$，这使得 A_t 和 S_{t+1} 的分布与策略无关：不管是 π 还是 b，所得条件期望都等于 $Q^\pi(s,a)$，可以正确地学习到目标函数。理解一下，意思就是随便什么样本，只要确实在状态 s 采取了动作 a 就可以用于更新 $Q(s,a)$，无关是采集的策略是谁。所以 off-policy 的算法学习 Q 更自然。

这就是为什么课件中总结称 “Values can only be estimated on-policy, Q-values off-policy”，严格来说这句话不普遍成立，但确实可以说 V 的直接 TD 估计通常是 on-policy 的，而 Q 也更容易用于 off-policy 算法。

至于反例当然存在，例如如果我们还是想学习 $V^\pi(s)$，但数据不是 π 而是 b 采集的，那通过**重要性采样**（importance sampling）校正后就可以做 off-policy 了，此处暂不展开；反过来，前文提过 off-policy 的算法本来就可以退化为 on-policy，包括上述学习 Q 的例子。

使用最优策略更新的 Q-Learning 是 off-policy 的，我们可以利用这一特性，为其添加一个**经验重放缓存**（experience replay buffer）。它存储最近 n 次状态转移中得到的数据样本，每次训练时会随机从中采集一个小批次（mini-batch）的数据，并且始终包括最近采

集的样本。这样做让部分旧样本仍参与学习，可减轻遗忘的问题，随机 mini-batches 也让数据分布更均匀、更接近 i.i.d.。

重放缓存的一个变体是**优先经验重放缓存**（prioritized experience replay buffer），它会优先存储误差较大的样本。直觉上，这是为了针对性反复学习之前发现没学好的样本，但同时这也有可能打乱训练数据分布（误差较大的样本可能在新旧、类型上分布不均），降低其对灾难性遗忘问题的抵抗力。

2.4.2. Exploration

训练或者说数据采集阶段，让智能体在环境中探索的最朴素的行动策略是纯**贪婪策略**（greedy policy），例如 Q-Learning 中就是取当前状态 Q 值较大的动作。这样做会产生一些问题，首先贪心策略时间一长就会反复走那几条高分路径，重放缓存中将被相似的样本塞满；用这些匮乏的样本训练将会使涉及不到的那些状态-动作对的 Q 值得不到可靠更新从而被“遗忘”，失去泛化能力；总是选取最值的贪婪策略还容易反复选取偶然产生的高分动作导致不断放大错误。

为缓解这些问题，引入**ϵ-贪婪探索策略**（ϵ-greedy exploration policy）：在探索时，以 ϵ 的概率进行随机游走，$1-\epsilon$ 的概率采取贪婪策略。同时这个 ϵ 将随时间线性衰减，直到 n_ϵ 步后或者低于某个阈值 ϵ 时停止，即前期较为随机、后期趋于贪心。

相较纯贪婪策略，ϵ-贪婪策略为探索引入了一定的随机性，令其一直保持探索的可能。当然，这是训练过程中采用的策略，实际部署用于测试时自然还是采用纯贪婪策略，即执行所学的最优策略。

2.4.3. Target Networks

本节还是关于 Section 2.3.2 和 Section 2.3.3 中老生常谈的参数估计自举问题。由前，半梯度法引入 θ' 以防止目标被同时更新，这里给用这套迟滞参数的模型（网络）起个正式名字叫**目标网络**（target networks）。

目标网络具体如何缓解自举问题就不重复了，前面没有细讲的一个问题是 θ' 和 θ 同步时机的问题。最直截了当的就是 Section 2.3.4 中实例的做法，只是截断了目标网络这一项的反向传播，这意味着每次前向传播采用的 θ 和 θ' 都是同步的，这样做其实还是有“目标和估计一起动”的问题存在。

要减轻该现象，最简单是可以让目标网络停住一阵子，给当前估计留一些追赶的时间。例如，每 $n=10$ 步才更新一下 $\theta' \leftarrow \theta$，在此之前都用旧值计算目标，这称为**硬目标更新**（hard target update）。

软目标更新则是类似于套一个低通滤波器，用：

$$\theta' \leftarrow (1-\eta)\theta' + \eta\theta \tag{75}$$

更新目标网络参数，η 可以选比如 0.1 之类，这被称为**软目标更新**（soft target update），总体来说效果更好一些。

2.5. Deep Q-Networks (DQN)

深度 Q 网络(DQN)包含一系列在线深度 Q-Learning 算法，总之就是用神经网络作为参数化模型，并且把上节的重放缓存和目标网络等手段都用上以增强稳定性的 Q-Learning。一轮（episode）更新的 PyTorch 实现框架如下：

```python
# Using mini-batch of transitions from replay buffer
batch = self.replay_buffer.sample()
targets = batch['rewards'] + self.gamma * (~batch['terminals'] \
        * self.target_q(batch['next_states']).max(dim = -1)[0])
values = self.q(batch['states']).gather(dim = -1, index = batch['actions'])

# Backpropagate loss
self.optimizer.zero_grad()
mse_loss(values, targets.detach()).backward()
self.optimizer.step()

# Update target network (hard or soft)
self.target_model_update()
```

接下来就举例说明关于 DQN 的一些性质。

首先是采样和更新频率对 DQN 表现的影响。我们不必要像前面讲的基础 Q-Learning 一样每采一个样本就更新一次梯度，而是可以一采就采一轮（包含多步）的数据，然后每轮更新 n 次梯度（n updates/episode）；或者一步一步采样，每 n 步采样更新一次梯度（sampling n steps/update）。这两种 n 放到同一个度量下大致就是反比的关系。

对于前者，n 越大训练越不稳定，即训练曲线的方差看起来越大，同时训练更快，但毕竟容易不稳定所以有限度。对于后者自然是相反，n 越大每次一起更新的样本越多，故训练越稳定，但相应学习速度变慢。

然后是关于学习阶段(phase)，在线训练过程中样本来自智能体的探索，故训练数据的分布不是固定的，智能体能访问到的状态会随着策略变化而改变，这在许多任务中会使学习过程自发地划分成多个阶段。如 Lunar Lander 模拟学习探测器降落的例子中，最初几乎是在随机、混乱地操作，直到某时刻学会了悬停才能稳定进入降落行为所在的状态空间区域。而进入新阶段的标志通常是损失函数不降反增，因为智能体在新的状态空间区域将学习到新的行为，此即课件总结的 “learning happens in phases, indicated by increasing losses”。

实践中 DQN 的调优需要经验和技巧。当出现失败（failure）情况，即策略无法生效时，通常可以考虑：

1. 提高梯度更新的学习率 α；
2. 提高 γ 以提升未来奖励的贡献；
3. 将网络输入归一化（零均值单位方差）；
4. 将奖励归一化到 $[-1, 1] \subset \mathbb{R}$（要谨慎）；
5. 提高探索时长；
6. 增加轮次大小；
7. 修改网络结构等。

当出现不稳定（instability）情况，即一段时间后策略不再进行有效的学习或提升时，可以考虑：
1. 降低学习率 α；
2. 降低 γ 减少错误的传播；
3. 减缓目标网络的更新；
4. 增加采样批次（mini-batch）大小；
5. 增加重放缓存大小；
6. 提高随机探索概率 ϵ 的最终下限等。

2.5.1. Why "Deep" Learning

TODO

2.5.2. Q-Learning extensions

TODO RL software architecture, Trust-region online bootstrapping, Distributional Q-learning, Modern DQN versions.

2.5.2.1. Double Q-Learning

在估计 Q 具有不确定性的情况下会累积出系统性的偏差：

$$\mathbb{E}\left[\max_a Q(s,a)\right] \geq \max_a \mathbb{E}[Q(s,a)] \tag{76}$$

具体一点说，如果估计的 Q 中存在噪声，先求最值再取平均的话平均过程无法消除最值选取中由于噪声导致的偏差，使得结果系统性高于先取平均再求最值的结果。对应到 Q-Learning 的实现中，我们每次取 Q 估计中的最大值构造目标样本进行更新，如果 Q 估计带有噪声最终就会产生偏差。

Double Q-Learning 的解决方案是将选取最大值的过程和构造目标的过程隔离，即分开维护两个 Q 表，Q' 用于最值选取，Q 用于构造目标，使得：

$$\mathbb{E}\left[Q\left(s, \arg\max_a Q'(s,a)\right)\right] \approx \max_a \mathbb{E}[Q(s,a)] \tag{77}$$

先前在 DQN 中引入了目标网络，算上原网络后天然就存在两个分开的网络。Deep Double Q-Learning（DDQN）用目标网络构造目标（Q），用原网络选取最优动作（Q'），在实践中有时会起到优化效果。

2.5.2.2. Dueling Network Architectures

可以利用优势函数，将网络架构从直接估计 Q 改为估计 V 和 A 的组合，即 $Q^\pi(s,a) = V^\pi(s) + A^\pi(s,a)$，可以更高效地学习状态价值。具体的原因例如，在动作集较大的情况下样本对 V 进行的更新会同步影响所有动作。架构的分解方式不固定，常用的例如：

$$Q_\theta(s,a) := V_\theta(s) + \left(A_\theta(s,a) - \max_{a \in \mathcal{A}} A_\theta(s,a)\right) \tag{78}$$

$$Q_\theta(s,a) := V_\theta(s) + \left(A_\theta(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a \in \mathcal{A}} A_\theta(s,a)\right) \tag{79}$$

2.5.3. Partial Observability and Deep Recurrent Q-Networks (DRQN)

现实中经常会出现系统中状态 s 不可观测的情况，考虑**部分可观马尔可夫决策过程**（partially observable Markov decision process，POMDP），相比普通 MDP 加入了取决于状态的**观测**（observations）变量 $o_t \sim O(\cdot \mid s_t) \in \mathcal{O}$，轨迹变为：

$$\mathrm{T} := \{O_0, A_0, R_0, O_1, ..., R_{t-1}, O_t\} \tag{80}$$

按照之前的假设，状态转移仍然是关于状态和动作的分布；奖励也依旧是关于状态和动作的确定函数，所以也可以不把奖励当随机变量处理：

$$\mathrm{T} := \{O_0, A_0, O_1, ..., A_{t-1}, O_t\} \in (\mathcal{O} \times \mathcal{A})^t \times \mathcal{O} \tag{81}$$

可以定义**信念分布**或称**状态认知分布**（belief distribution）：

$$b_t(s \mid \tau_t) := \mathbb{P}(S_t = s \mid o_0, a_0, ..., o_t) \tag{82}$$

其中 τ_t 是估计到 o_t 为止的样本，同时依旧将条件中的随机变量等简写为样本。我们希望 b 表达的是智能体对 t 时刻隐含状态分布的一个估计。课件里是没有写 b_t 的下标 t 的，而这里我们将它作为参数显式标注出来了，不然定义里的 t 就没有来源，只能从 τ_t 的下标勉强看出；而且 τ_t 的长度也随 t 变化，b_t 本身就是不同时刻各自的一系列分布而不是一个分布。

套用贝叶斯估计框架可以得到如下状态转移（和习惯不同，这里 s' 是前一个状态）：

$$b_{t+1}(s \mid \tau_{t+1}) \propto O(o_{t+1} \mid s) \int P(s \mid s', a_t) b_t(s' \mid \tau_t) \, \mathrm{d}s' \tag{83}$$

关于信念分布状态转移的贝叶斯推导：

由贝叶斯定理以及前述定义有：

$$\begin{aligned}
b_{t+1}(s_{t+1} \mid \tau_{t+1}) &= \mathbb{P}(S_{t+1} = s_{t+1} \mid o_0, a_0, ..., o_{t+1}) \\
&= \mathbb{P}(s_{t+1} \mid \tau_t, a_t, o_{t+1}) \\
&\propto \mathbb{P}(o_{t+1} \mid s_{t+1}, \cancel{\tau_t, a_t}) \mathbb{P}(s_{t+1} \mid \tau_t, a_t) \\
&= \mathbb{P}(o_{t+1} \mid s_{t+1}) \mathbb{P}(s_{t+1} \mid \tau_t, a_t) \\
&= \mathbb{P}(o_{t+1} \mid s_{t+1}) \int \mathbb{P}(s_{t+1}, s_t \mid \tau_t, a_t) \, \mathrm{d}s_t \\
&= \mathbb{P}(o_{t+1} \mid s_{t+1}) \int \mathbb{P}(s_{t+1} \mid s_t, \cancel{\tau_t}, a_t) \mathbb{P}(s_t \mid \tau_t, \cancel{a_t}) \, \mathrm{d}s_t \\
&= O(o_{t+1} \mid s_{t+1}) \int P(s_{t+1} \mid s_t, a_t) b_t(s_t \mid \tau_t) \, \mathrm{d}s_t
\end{aligned} \tag{84}$$

把 s_{t+1} 重命名成 s，s_t 重命名成 s' 就是 Equation 83 了。这个过程其实就是贝叶斯估计中标准的预测和更新框架。

有了对状态分布的估计，我们就可以将前述 POMDP 转化为一个等价的基于 belief 的 MDP。历史 τ_t 可以直接作为这个 MDP 的状态，因为它包含了所有历史信息，是对 POMDP 状态的充分统计量（sufficient statistic），即知道 τ_t 就可以计算 belief b_t。

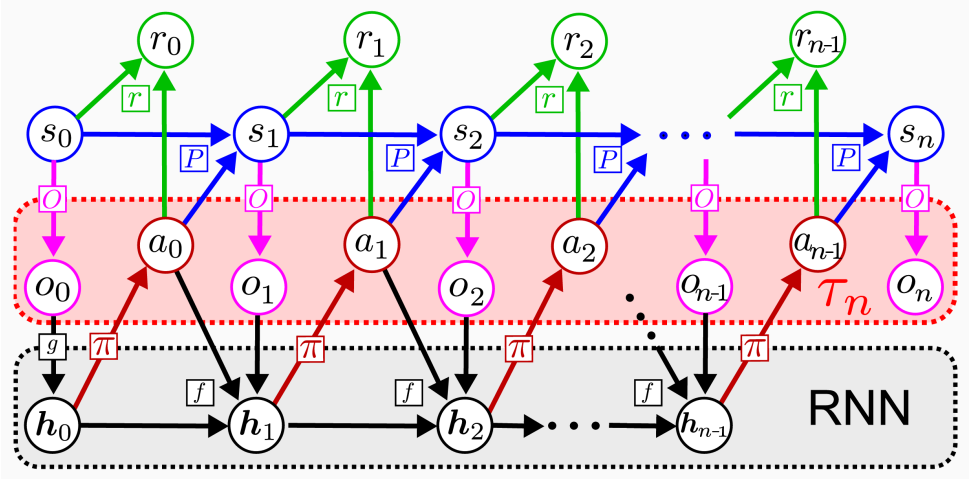


Figure 2: 部分可观马尔可夫链及 RNN 历史编码示意图（来自 TU Delft 课件）

如 Figure 2 所示，其中 $\boldsymbol{h}_t$ 对应的就是历史 τ_t。参考图中结构很容易补全这个等价 MDP 的各要素，其奖励函数为：

$$r(\tau_t, a) = \int r(s, a) b(s \mid \tau_t) \, \mathrm{d}s \tag{85}$$

状态转移概率为：

$$P(\tau_{t+1} \mid \tau_t, a_t) = \iint O(o_{t+1} \mid s') P(s' \mid s, a_t) b_t(s \mid \tau_t) \, \mathrm{d}s \, \mathrm{d}s' \tag{86}$$

还可以写出最优动作值函数为：

$$Q^*(\tau_t, a) = r(\tau_t, a) + \gamma \int P(\tau_{t+1} \mid \tau_t, a) \max_{a'} Q^*(\tau_{t+1}, a') \, \mathrm{d}\tau_{t+1} \tag{87}$$

历史信息毕竟是一长串变量，而且长度可变，实际实现中将 τ_t 直接作为状态不现实，同时考虑到马尔可夫链是一个时序系统，我们可以用 RNN 把历史 τ_t 编码成固定长度的向量 $\boldsymbol{h}$，即用 RNN 的隐藏状态 $\boldsymbol{h}_t$ 作为 belief 的近似：

$$\boldsymbol{h}_t = f(\boldsymbol{h}_{t-1}, o_t, a_{t-1}) \tag{88}$$

就如 Figure 2 中所示，循环神经元的个数就是 $\boldsymbol{h}$ 的长度。这套框架可以无缝接入大部分强化学习算法，实现端到端学习，即直接把 RNN 的输出代替状态 s 接到 RL 输入。而且不需要 POMDP 的模型先验信息，但也因此不保证泛化性能，即 RNN 本身也可能学不到完整的状态空间。

既然都用 RNN 了，同样思路下自然也可以把 RNN 替换成 LSTM 来编码 τ_t：

$$\begin{aligned}\underbrace{\boldsymbol{h}_t, \boldsymbol{c}_t}_{\text{encoded } \tau_t} &= \text{LSTM}\left(\underbrace{\boldsymbol{h}_{t-1}, \boldsymbol{c}_{t-1}}_{\text{encoded } \tau_{t-1}}, a_{t-1}, o_t\right) \in \mathbb{R}^m, \\ \boldsymbol{h}_0, \boldsymbol{c}_0 &= \text{LSTM}\left(\underbrace{\mathbf{0}, \mathbf{0}}_{\text{init}}, \underbrace{0, o_0}_{\text{first}}\right)\end{aligned} \tag{89}$$

在这套框架上我们可以基于这套“历史信息编码状态”实现端到端 Q-Learning，使用的损失函数为：

$$\mathcal{L}[\theta] = \mathbb{E}\left[\sum_{t=0}^{n}\left(r_t + \gamma \max_a q_{\theta'}(\boldsymbol{h}_{t+1}, \boldsymbol{c}_{t+1}, a) - q_\theta(\boldsymbol{h}_t, \boldsymbol{c}_t, a_t)\right)^2 \middle| \tau_n \sim \mathcal{D}\right] \tag{90}$$

这就是**深度循环 Q 网络**（deep recurrent Q-networks，DRQN）算法。相比 DQN，由于引入了 RNN/LSTM，重放缓存等模块需要进行一些修改：

1. 采样批次中的样本不再是状态转移（transistions）而应该是连续的片段（episodes），因为端到端训练中同时学习 LSTM 和 Q 的参数，而前者需要历史结构来正确学习编码历史。
2. 片段可能长度不一，所以在连续定长存储时需要采用补零填充等技巧，同时要注意填充的这部分不应该加入损失计算以防污染梯度。

用 PyTorch 实现节选如下：

```python
import torch as th

batch = self.replay_buffer.sample()
mask = batch['mask'][:-1]

# compute all recurrent Q-values with function q
first_act = th.zeros(1, *batch['actions'].shape[1:]) # a_(-1) = 0
delayed_acts = th.cat([first_act, batch['actions'][:-1]], dim = 0)
values = q(th.cat([delayed_acts, batch['observations']], dim = -1))[0]

# derive the regression targets from the values
max_v = ~batch['terminals'] * values.max(dim = -1, keepdim = True)[0]
targets = (batch['rewards'][:-1] + gamma * max_v[1:]) * mask

# derive the MSE loss to be optimized
current_values = values[:-1].gather(dim = -1, index = batch['actions'][:-1]) * mask
loss = mse_loss(current_values, targets.detach(), reduction = 'sum')

# gradient descent step on supervised regression loss
optimizer.zero_grad()
(loss / mask.sum()).backward()
optimizer.step()
```

举个实际的例子，考虑迷宫探索任务，智能体只能观察到第一人称视角的摄像头画面，每步可以选择前进或左右旋转一定角度，需要收集一些奖励物品并抵达终点，要求为 DQN 算法设计一个神经网络架构。

这个问题中没有一个显式的、可观测的状态，考虑通过 LSTM 学习一个。可以猜测这个学到的隐含状态可能会表示智能体在迷宫中的位置和朝向、奖励和目标的相对位置、对环境的部分记忆等。

一个合理的网络架构是先通过 CNN 等将输入图片特征进行编码，送入 LSTM 学习历史编码，再送入参数化 Q 网络，最后输出该输入（对应隐含状态）下各动作的价值函数值。

3. Policy Optimization and On-Policy Actor-Critic

考虑与先前学习值函数的方式不同的另一条思路，直接学习策略分布。定义参数化的策略 $\pi_\theta(a \mid s)$，依旧最优化总体期望回报 $J^{\pi_\theta} := \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R_t\right]$。先考虑最朴素的方法，即直接求 J^{π_θ} 的梯度用于梯度上升。

这一类优化策略的方法可以按特点归纳到一张谱图上，如 Figure 3 所示，本章节先讨论左侧三类方法。顺便，图中 TRPO/PPO 被划分在了 off-policy loss 中，这是不太常见的，虽然 TRPO/PPO 会多次使用部分更新前策略的样本，但由于更新幅度小一般还是算作 on-policy 算法，所以放到该章节中，详见后文。

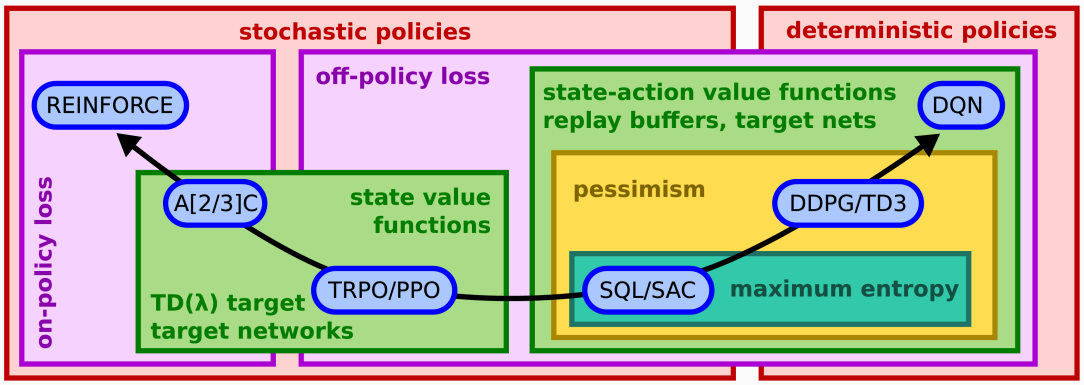


Figure 3: 策略梯度类算法谱图（来自 TU Delft 课件）

3.1. Parameterized Stochastic Policies

我们采用的策略是**随机策略**（stochastic policy），即参数化的是一个概率分布，而不同于 DQN 中直接通过参数化价值函数确定（deterministic）的策略。原因在于，下节中会推导策略梯度定理，其中涉及（对数）策略分布的梯度，如果采用确定策略就意味着策略分布退化为 δ 分布，梯度的质量也就会很一般。下个章节的 DDPG、TD3 等算法会基于确定策略展开。

常用的随机策略分布通常也就那几种，离散动作的情况下多采用 **Softmax 策略**：

$$\pi_\theta(a \mid s) = \frac{\exp(z_\theta(s,a))}{\sum_{a'} \exp(z_\theta(s,a'))} \tag{91}$$

非常常见，分布的结构固定后其中的分布参数 z_θ 等可以再用其他模型（如神经网络）进行参数化。Softmax 策略对应的对数梯度为：

$$\begin{aligned}
\nabla_\theta \ln \pi_\theta(a \mid s) &= \nabla_\theta \left[z_\theta(s,a) - \ln \sum_{a'} \exp(z_\theta(s,a')) \right] \\
&= \nabla_\theta z_\theta(s,a) - \frac{\sum_{a'} \exp(z_\theta(s,a')) \nabla_\theta z_\theta(s,a')}{\sum_{a'} \exp(z_\theta(s,a'))} \\
&= \nabla_\theta z_\theta(s,a) - \sum_{a'} \pi_\theta(a' \mid s) \nabla_\theta z_\theta(s,a')
\end{aligned} \tag{92}$$

也就是动作 a 的策略梯度减去所有动作策略梯度按策略分布加权的平均值。

对于连续动作空间则常采用**高斯策略**，设动作维度为 d：

$$
\begin{aligned}
\pi_\theta(a \mid s) &= \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s)) \\
&= \frac{1}{(2\pi)^{d/2}|\Sigma_\theta(s)|^{1/2}} \exp\left(-\frac{1}{2}(a-\mu_\theta(s))^\top \Sigma_\theta^{-1}(s)(a-\mu_\theta(s))\right)
\end{aligned} \tag{93}
$$

对应的对数梯度推导略（ TODO ）。

3.2. Policy Gradient Theorem

对于随机轨迹 $\mathrm{T} = \{S_0, A_0, R_0, S_1, ..., R_{n-1}, S_n\}$，我们之前定义过整条轨迹的折扣回报 Equation 8，这里我们再定义从某个指定时刻 t 作为起点的折扣回报：

$$
G_t := \sum_{k=0}^{\infty} \gamma^k R_{t+k} \tag{94}
$$

课件上这里用的符号是 R_t，但我们这里大写 R_t 已经用于表示奖励的随机变量，为避免混淆采用 $G_t := G_t(\mathrm{T})$，样本记为 $g_t := g_t(\tau)$。于是总期望回报又可写成 $J^{\pi_\theta} := \mathbb{E}_{\pi_\theta}[G_0]$。在继续推导前，我们先将轨迹分布显式写出为：

$$
p_{\mathrm{T}}^{\pi_\theta}(\mathrm{T}) = \rho(S_0) \prod_{t=0}^{n-1} \pi_\theta(A_t \mid S_t) P(S_{t+1} \mid S_t, A_t) \tag{95}
$$

其对数梯度为：

$$
\begin{aligned}
\nabla_\theta \ln p_{\mathrm{T}}^{\pi_\theta}(\mathrm{T}) &= \nabla_\theta \left[\ln \rho(S_0) + \sum_{t=0}^{n-1} \left[\ln \pi_\theta(A_t \mid S_t) + \ln P(S_{t+1} \mid S_t, A_t) \right] \right] \\
&= \nabla_\theta \sum_{t=0}^{n-1} \ln \pi_\theta(A_t \mid S_t)
\end{aligned} \tag{96}
$$

接下来推导总期望回报的梯度：

$$
\begin{aligned}
\nabla_\theta J^{\pi_\theta} &= \nabla_\theta \mathbb{E}_{\pi_\theta}[G_0] \\
&= \nabla_\theta \sum_{\tau} g_0(\tau) p_{\mathrm{T}}^{\pi_\theta}(\tau) \\
&= \sum_{\tau} g_0(\tau) [\nabla_\theta p_{\mathrm{T}}^{\pi_\theta}(\tau)] \\
&= \sum_{\tau} g_0(\tau) [p_{\mathrm{T}}^{\pi_\theta}(\tau) \nabla_\theta \ln p_{\mathrm{T}}^{\pi_\theta}(\tau)] \\
&= \sum_{\tau} [g_0(\tau) \nabla_\theta \ln p_{\mathrm{T}}^{\pi_\theta}(\tau)] p_{\mathrm{T}}^{\pi_\theta}(\tau) \\
&= \mathbb{E}_{\pi_\theta}[G_0 \nabla_\theta \ln p_{\mathrm{T}}^{\pi_\theta}(\mathrm{T})] \\
&= \mathbb{E}_{\pi_\theta}\left[G_0 \nabla_\theta \sum_{t=0}^{n-1} \ln \pi_\theta(A_t \mid S_t) \right] \\
&= \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{n-1} G_0 \nabla_\theta \ln \pi_\theta(A_t \mid S_t) \right]
\end{aligned} \tag{97}
$$

到这一步实际上已经可以用了（用轨迹样本估计这个期望），但我们还可以把估计的方差降低一些。直觉上来说，由于之后的动作和之前的奖励无关，所以 G_0 中 t 时刻之前的部分是可以挪出去的。我们还可以证明挪出去这部分期望为零，可以直接消去，具体地，将 G_0 拆分为：

$$\begin{aligned} G_0 \coloneqq \sum_{k=0}^{\infty} \gamma^k R_k &= \sum_{k=0}^{t-1} \gamma^k R_k + \sum_{k=t}^{\infty} \gamma^k R_k \\ &= \sum_{k=0}^{t-1} \gamma^k R_k + \gamma^t \underbrace{\sum_{k=0}^{\infty} \gamma^k R_{k+t}}_{G_t} \\ &=: C_t + \gamma^t G_t \end{aligned} \tag{98}$$

于是之前的结果可拆分整理为：

$$\nabla_\theta J^{\pi_\theta} = \sum_{t=0}^{n-1} \mathbb{E}_{\pi_\theta}[C_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t)] + \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{n-1} \gamma^t G_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \tag{99}$$

记 $H_t = \{S_0, A_0, R_0, S_1, ..., R_{t-1}, S_t\}$，对上式第一项每项有：

$$\begin{aligned} \mathbb{E}_{\pi_\theta}[C_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t)] &\overset{(1)}{=} \mathbb{E}_{H_t}\left[\mathbb{E}_{A_t \sim \pi_\theta(\cdot|S_t)}[C_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t) \mid H_t]\right] \\ &\overset{(2)}{=} \mathbb{E}_{H_t}\left[C_t \mathbb{E}_{A_t \sim \pi_\theta(\cdot|S_t)}[\nabla_\theta \ln \pi_\theta(A_t \mid S_t) \mid H_t]\right] \end{aligned} \tag{100}$$

上式中第 (1) 步是全概率公式将轨迹分布分解为前半段和 H_t 和 A_t（t 时刻之后的随机变量在期望中没有用到所以都可以消去）；第 (2) 步是由于给定 H_t 时 C_t 可视为常数，可提取到期望之外。而其中：

$$\begin{aligned} \mathbb{E}_{A_t \sim \pi_\theta(\cdot|S_t)}[\nabla_\theta \ln \pi_\theta(A_t \mid S_t) \mid H_t] &= \sum_{a_t} \nabla_\theta \ln \pi_\theta(a_t \mid S_t), \quad S_t \text{ given by } H_t \\ &= \sum_{a_t} \pi_\theta(a_t \mid S_t) \nabla_\theta \ln \pi_\theta(a_t \mid S_t) \\ &= \nabla_\theta \sum_{a_t} \pi_\theta(a_t \mid S_t) \\ &= 0 \end{aligned} \tag{101}$$

于是总期望回报式中关于 C_t 的项为 0，最终有：

$$\nabla_\theta J^{\pi_\theta} = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty} \gamma^t G_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \tag{102}$$

即**策略梯度定理**（Policy Gradient Theorem）。这个式子可以直观上理解一下，$\nabla_\theta \ln \pi_\theta(A_t \mid S_t)$ 是“可以提高 S_t 下选动作 A_t 概率的参数更新方向”，$\gamma^t G_t$ 是采取该动作后所得相对初始时刻的折扣回报，所以总期望回报的参数更新方向就是把所有时刻这些子更新方向按回报加权求和并取期望。再具体一点，某个动作带来的折扣回报越大，提高该动作概率的参数优化方向就会在总优化方向中占比越大，优化后选取该动作的概率也就越大。

3.2.1. REINFORCE

REINFORCE 算法的核心思路就是直接用采样所得轨迹样本去估计这个期望，从而更新策略。我们要最大化总期望回报，即最小化损失函数 $\mathcal{L}_{\pi_\theta}[\theta] := -J^{\pi_\theta}$，梯度为：

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] &= -\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t G_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \\ &\approx \nabla_\theta\left[-\frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{n-1}\gamma^t g_t^i \ln \pi_\theta(a_t^i \mid s_t^i)\right]\end{aligned} \tag{103}$$

REINFORCE 是典型的 on-policy 策略梯度算法，直接使用当前策略采样并更新，策略用概率分布表示，是随机策略。其简单直接，不需要估计价值函数，缺点则是样本利用效率低（朴素 on-policy 算法的通病，策略更新后旧样本不再能使用），估计方差较大（见后）。

如 Figure 3 所示，之前的 DQN 是与其相对的另一个极端 off-policy value-based 算法，完全依赖动作值函数估计，策略是通过最大化动作值函数得到的**确定策略**（deterministic policy）。由于可以复用，样本利用率高，但是在很难学出一个泛化性优秀的价值函数时就容易不稳定。

3.3. Actor-Critic Algorithms

回到 REINFORCE，我们提到它的估计方差（通常）比较大，主要来源于样本回报中的随机性：

$$g_t^i = \sum_{j=0}^{\infty}\gamma^j r_{t+j}^i \tag{104}$$

同样都是无偏估计量，在相同样本数量下方差较小的估计量估计更稳定（每个样本更可信），这是我们希望降低估计量方差的原因。

之前我们从 G_0 中删掉了 C_t，这是各种可能性中与未来无关的共同部分，通常可以降低方差，现在我们考虑是否还有其他共同部分。如前所述，我们希望衡量的是采取各动作后结果的优劣，从而将其作为权重加到各更新方向上，所以仅体现出相对优劣即可。考虑从 G_t 中减去一个 baseline B_t：

$$\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t (G_t - B_t)\nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \tag{105}$$

当然，首先要保证这么做之后估计还是无偏的，借用 Equation 100 处的 H_t 定义，我们要求选取的 baseline 只能和 H_t（链上 S_t 及之前的随机变量）有关，记为 $B_t := b(H_t)$。和前面证明 C_t 项为零的思路基本一致：

$$
\begin{aligned}
&\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t b(H_t)\nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \\
&= \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{\pi_\theta}[b(H_t)\nabla_\theta \ln \pi_\theta(A_t \mid S_t)] \\
&= \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{H_t}\left[\mathbb{E}_{A_t\sim\pi_\theta(\cdot|S_t)}[b(H_t)\nabla_\theta \ln \pi_\theta(A_t \mid S_t) \mid H_t]\right] \\
&= \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{H_t}\left[b(H_t)\mathbb{E}_{A_t\sim\pi_\theta(\cdot|S_t)}[\nabla_\theta \ln \pi_\theta(A_t \mid S_t) \mid H_t]\right] \\
&= 0
\end{aligned}
\tag{106}
$$

这里第一步将无限时域求和提到期望外面，严格来说需要满足一些有界性条件，在我们的问题假设下（例如奖励有界、折扣因子等）基本都成立，此处省略。第二步是条件期望分解，$\mathbb{E}_{\pi_\theta}$ 简写的随机变量是整条轨迹，由于期望内部不涉及 S_t、A_t 以后的随机变量，可以将它们舍去，剩下的就可以分解为 H_t（到 S_t 为止的轨迹）和 A_t 两层。最后为零的项来自 Equation 101。

该式为零也就说明在 G_t 上减去任意仅和 H_t 有关的 baseline 都不破坏估计的无偏性。我们通常选取值函数估计 $V^{\pi_\theta}(S_t)$ 当 baseline，它仅和当前状态 S_t 有关，符合条件。选择值函数作为 baseline 是符合直觉的，$G_t - V^{\pi_\theta}(S_t)$ 可以解读为在该状态下采取某动作所得回报比“平均情况下在该状态所得回报”好多少，相当于做了一个归一化。这个对“好多少”的衡量也被称为**优势**（advantage），对应**优势函数**（advantage function）：

$$
A^\pi(s,a) := Q^\pi(s,a) - V^\pi(s) \tag{107}
$$

> **关于 baseline 的选取与对方差的优化：**
>
> 此外，选值函数直观简单，但在“最小化梯度方差”这一目标下并非最优。TODO 到底怎么降低的方差。

接下来的问题是 REINFORCE 框架下并没有估计值函数，所以需要一个分支负责估计值函数。值函数是对当前策略价值的衡量，评估一个策略的好坏，所以该分支称为**批评者**（critic），维护一个参数化的**值函数估计** $v_\phi(s) \approx V^{\pi_\theta}(s)$；原本负责探索、采样的分支则称为**执行者**（actor），并用**对优势的估计**进行策略更新。

此即 Actor-Critic 名称的由来，优势估计和值函数估计也是 Actor-Critic 架构下最关键的两个问题。

3.3.1. Value Function Estimation

我们先讨论值函数估计的具体方法，即 critic 如何学习 $v_\phi(s)$。基于采样所得轨迹 $\tau_\infty = \{s_t, a_t, r_t\}_{t=0}^{\infty}$，采用均方误差损失函数：

$$\mathcal{L}[\phi] \coloneqq \sum_{t=0}^{\infty} \left(\overbrace{\underbrace{y_t(\tau_\infty)}_{\text{targets}} - \underbrace{v_\phi(s_t)}_{\text{value}}}^{\text{advantage}} \right)^2 \tag{108}$$

其中目标（targets）的选择有很多种。规范起见，定义轨迹样本从 l 时刻到 r 时刻的切片：

$$\tau_{l:r} = \{s_l, a_l, r_l, s_{l+1}, ..., r_{r-1}, s_r\} \tag{109}$$

顺便定义 $\tau_r \coloneqq \tau_{:r} \coloneqq \tau_{0:r}$ 表示从头开始到 r 结束的片段，τ_∞ 也可以纳入其中。

当手上有一段长度为 n 的轨迹切片 $\tau_{t:t+n}$ 时，最自然的想法就是把这些样本全都利用上。前 n 步有样本就用样本计算，余下的远未来项用值函数估计，此即多步目标（n-step targets）：

$$y_t^n(\tau_{t:t+n}) \coloneqq \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n v_\phi(s_{t+n}) \tag{110}$$

课件中括号里写的是 τ_n 而非 $\tau_{t:t+n}$，只表示了长度容易混淆，这里我们明确写出切片范围。这个 $\tau_{l:r}$ 只是用来标注一下定义 y_t^n 所需的变量，实际起点和长度取决于 y_t^n 的上下标，简单起见有时也会写包含所有信息的 τ_∞，或者直接省略括号内的参数。

计算可以得知对于任意 $n \in \mathbb{N}$，样本来自 π 的多步目标的期望都等于 $V^\pi(s_t)$：

$$\begin{aligned}
\mathbb{E}_\pi[y_t^n \mid s_t] &= \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k} + \gamma^n v_\phi(S_{t+n}) \,\middle|\, S_t = s_t \right] \\
&\overset{!}{=} \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k} + \gamma^n V^\pi(S_{t+n}) \,\middle|\, S_t = s_t \right] \\
&= \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k} + \gamma^n \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{k+t+n} \,\middle|\, S_{t+n}, ..., S_t \right] \,\middle|\, S_t = s_t \right] \\
&= \mathbb{E}_\pi \left[\sum_{k=0}^{n-1} \gamma^k R_{t+k} + \sum_{k=n}^{\infty} \gamma^k R_{t+k} \,\middle|\, S_t = s_t \right] \\
&= \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{k+t} \,\middle|\, S_t = s_t \right] \\
&= V^\pi(s_t)
\end{aligned} \tag{111}$$

注意到当 $n = 1$ 时，$y_t^1(\tau_\infty) \coloneqq r_t + \gamma v_\phi(s_{t+1})$ 就是常用的**时序差分**（temporal difference，TD）目标，特点是收敛慢（迭代次数多）而方差较小；当所用样本足够长时就是 **Monte-Carlo** 目标，特点是收敛快（迭代次数少）但方差较大：

$$y_t^{\mathrm{MC}}(\tau_\infty) \coloneqq \lim_{n \to \infty} y_t^n(\tau_\infty) \tag{112}$$

现在问题来了，TD 目标和 MC 目标各有优劣，如何综合二者的特点？直接使用多步目标虽然是一种折中方案，但固定了样本长度，并没有实现综合考虑短期和长期目标的效果。考虑无限长样本 τ_∞，我们可以对所有长度 $[1, \infty)$ 的多步目标进行加权平均，选一个系数 $\lambda \in [0, 1]$，权重就采用几何级数 λ^k，这称为 **TD(λ) 目标**：

$$y_t^{\mathrm{TD}(\lambda)}(\tau_\infty) := (1-\lambda)\sum_{k=0}^{\infty}\lambda^k y_t^{k+1}(\tau_\infty) \tag{113}$$

其中 $1-\lambda$ 是归一化因子，用以确保 $\sum_{k=0}^{\infty}\lambda^k = \frac{1}{1-\lambda}$ 乘上它后能将权重和化为 1。λ 较小时级数衰减较快，较长的多步目标权重就低，对应地就偏向短期目标。可以验证，$\lambda \to 0$ 时它就是 TD 目标 y_t^1，$\lambda \to 1$ 时它就是 Monte-Carlo 目标 y_t^{MC}（TODO）。

3.3.2. Advantage Actor-Critic (A2C)

接下来讨论优势估计的具体方法。Actor-Critic 只是架构的名称，在我们选取值函数作为 baseline，将其与回报期望组合为“优势”进行估计时，实际上已经算是 **Advantage Actor-Critic** 即 A2C 算法。朴素的优势估计使用 $g_t^i - v_\phi(s_t^i)$，对应样本损失函数：

$$\mathcal{L}'_{\pi_\theta}[\theta] := -\frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{n-1}\gamma^t \ln \pi_\theta(a_t^i \mid s_t^i)\left[g_t^i - v_\phi(s_t^i)\right] \tag{114}$$

需要注意的是这里的 g_t^i 需要通过第 i 条轨迹样本的连续片段计算得到，故这种优势估计是基于轨迹片段（episodes）的 **Monte-Carlo** 估计。这种估计是无偏的，且因为样本信息量大，训练较快；但整条的轨迹包含大量的随机性，导致样本方差通常很大，这部分方差无法靠减去一个 baseline 优化。

为此，顺着经典思路，我们考虑与 Monte-Carlo 估计相对的 **TD** 估计。具体地，假设 $v_\phi(s_{t+1}^i) \approx V^{\pi_\theta}(s_{t+1}^i)$，即允许用当前估计近似未来目标，就可以用 TD 估计作为优势估计，得到相应的样本损失函数：

$$\mathcal{L}''_{\pi_\theta}[\theta] := -\frac{1}{m}\sum_{i=1}^{m}\sum_{t=0}^{n-1}\gamma^t \ln \pi_\theta(a_t^i \mid s_t^i)\left[r_t^i + \gamma v_\phi(s_{t+1}^i) - v_\phi(s_t^i)\right] \tag{115}$$

由于引入 bootstrapping，这个估计是有偏的，但因为单个样本短，能显著降低方差。

回顾上节，值函数估计中我们引入 TD(λ) 目标实现 MC 和 TD 的综合，优势估计中自然也有类似的方案，即**广义优势估计**（generalized advantage estimation，GAE）：

$$\hat{A}_t^{\mathrm{GAE}(\lambda),n}(\tau_n) := \frac{1-\lambda}{\lambda - \lambda^{n-t+1}}\sum_{k=1}^{n-t}\lambda^k\left(y_t^k(\tau_{t:t+k}) - v_\phi(s_t)\right) \tag{116}$$

课件中用的符号是 G_t^λ，但我们之前用 G 表示折扣回报了，故换成 $\hat{A}$，更直白地说明它是对优势函数的估计。这里我们考虑的样本片段 τ_n 到 s_n 就结束的序列，所以求和索引 k 的上限就到 $n-t$，确保 $y_t^k(\tau_{t:t+k})$ 有定义；如果是无限样本长度（$n \to \infty$）就都是 ∞ 了，可以直接省去上标中的 n。

从公式角度直观看一下这个式子，其实就是用多步目标计算的优势 $y_t^k(\tau_{t:t+k}) - v_\phi(s_t)$ 用几何级数 λ^k 加权平均，前面是归一化系数。这和 TD(λ) 思路是完全一样的，只是一个在值函数估计里，一个在优势估计里。

关于另一种常见的 GAE 定义及两种定义的关联性：

在其他材料中 GAE 经常定义为：

$$\hat{A}_t^{\mathrm{GAE}(\lambda),n}(\tau_n) := \sum_{l=0}^{n-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t := r_t + \gamma v_\phi(s_{t+1}) - v_\phi(s_t) \tag{117}$$

这种写法是将 GAE 写成了 TD 优势 δ_t 的加权和形式，形式上简洁一些。我们希望可以证明 Equation 116 和 Equation 117 里的两种定义是一致的，但先说结论：在无限长样本下确实如此，但**样本长度有限时二者并不等价**。

先计算优势：

$$\begin{aligned}
& y_t^k - v_\phi(s_t) \\
=& \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k v_\phi(s_{t+k}) - v_\phi(s_t) \\
=& \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \gamma^k v_\phi(s_{t+k}) - v_\phi(s_t) + \sum_{l=1}^{k-1} \gamma^l v_\phi(s_{t+l}) - \sum_{l=1}^{k-1} \gamma^l v_\phi(s_{t+l}) \\
=& \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \sum_{l=1}^{k} \gamma^l v_\phi(s_{t+l}) - \sum_{l=0}^{k-1} \gamma^l v_\phi(s_{t+l}) \\
=& \sum_{l=0}^{k-1} \gamma^l r_{t+l} + \sum_{l=0}^{k-1} \gamma^l \cdot \gamma v_\phi(s_{t+l+1}) - \sum_{l=0}^{k-1} \gamma^l v_\phi(s_{t+l}) \\
=& \sum_{l=0}^{k-1} \gamma^l \left(r_{t+l} + \gamma v_\phi(s_{t+l+1}) - v_\phi(s_{t+l}) \right) \\
=& \sum_{l=0}^{k-1} \gamma^l \delta_{t+l}
\end{aligned} \tag{118}$$

代入 Equation 116 有：

$$\begin{aligned}
& \frac{1-\lambda}{\lambda - \lambda^{n-t+1}} \sum_{k=1}^{n-t} \lambda^k \left(y_t^k - v_\phi(s_t) \right) \\
=& \frac{1-\lambda}{\lambda - \lambda^{n-t+1}} \sum_{k=1}^{n-t} \lambda^k \sum_{l=0}^{k-1} \gamma^l \delta_{t+l} \\
=& \frac{1-\lambda}{\lambda - \lambda^{n-t+1}} \sum_{l=0}^{n-t-1} \sum_{k=l+1}^{n-t} \lambda^{k-l} \lambda^l \gamma^l \delta_{t+l} \\
=& \sum_{l=0}^{n-t-1} (\lambda\gamma)^l \delta_{t+l} \cdot \frac{1-\lambda}{\lambda - \lambda^{n-t+1}} \sum_{k=l+1}^{n-t} \lambda^{k-l} \\
=& \sum_{l=0}^{n-t-1} (\lambda\gamma)^l \delta_{t+l} \cdot \underbrace{\frac{1-\lambda}{\lambda - \lambda^{n-t+1}} \sum_{k=1}^{n-t-l} \lambda^k}_{C(l)}
\end{aligned} \tag{119}$$

关键步骤就是交换了一下求和顺序，画一下 k-l 平面内求和的三角形区域就行。不过结果很遗憾，两种定义形式并非完全一致，而且 $C(l)$ 与 l 相关，二者也不是简单的比例关系。但在 $n\to\infty$ 时，由于有 $n-t-l=n-t+1\to\infty$，系数 $C(l)$ 就等于 1 了，即有：

$$\hat{A}_t^{\mathrm{GAE}(\lambda)}(\tau_\infty)=\sum_{l=0}^{\infty}(\lambda\gamma)^l\delta_{t+l} \tag{120}$$

Equation 116 的定义更直观，而 Equation 117 实际上通常也是以 $n\to\infty$ 的形式出现，之后我们主要以前者为主。

按照预期，我们可以验证在两个极端情况下 GAE 分别应该接近 TD 估计和 MC 估计。当 $\lambda\to 0$ 时：

$$\begin{aligned}\lim_{\lambda\to 0}\hat{A}_t^{\mathrm{GAE}(\lambda),n}(\tau_n)&=y_t^1(\tau_n)-v_\phi(s_t)\\&=r_t+\gamma v_\phi(s_{t+1})-v_\phi(s_t)\end{aligned} \tag{121}$$

这就是 TD 估计，对应 Equation 115。当 $\lambda\to 1$ 时，应用洛必达法则：

$$\begin{aligned}\lim_{\lambda\to 1}\hat{A}_t^{\mathrm{GAE}(\lambda),n}(\tau_n)&=\lim_{\lambda\to 1}\frac{1-\lambda}{\lambda-\lambda^{n-t+1}}\sum_{k=1}^{n-t}\lambda^k\big(y_t^k(\tau_{t:t+k})-v_\phi(s_t)\big)\\&=\lim_{\lambda\to 1}\frac{\frac{\mathrm{d}}{\mathrm{d}\lambda}\left[(1-\lambda)\sum_{k=1}^{n-t}\lambda^{k-1}\big(y_t^k(\tau_{t:t+k})-v_\phi(s_t)\big)\right]}{\frac{\mathrm{d}}{\mathrm{d}\lambda}[1-\lambda^{n-t}]}\\&=\frac{1}{n-t}\sum_{k=1}^{n-t}\big(y_t^k-v_\phi(s_t)\big)\end{aligned} \tag{122}$$

或者直接从定义角度，$\lambda\to 1$ 时权重都相同，所以就是求等权平均，直接写出系数得到结果。注意它**不等于** MC 估计，**但**当 $n\to\infty$ 时它就收敛到 MC 估计，用 Equation 120 推导比较方便：

$$\begin{aligned}\hat{A}_t^{\mathrm{GAE}(1)}(\tau_\infty)&=\sum_{l=0}^{\infty}\gamma^l\delta_{t+l}\\&=\sum_{l=0}^{\infty}\gamma^l\big(r_{t+l}+\gamma v_\phi(s_{t+l+1})-v_\phi(s_{t+l})\big)\\&=\sum_{l=0}^{\infty}\gamma^l r_{t+l}+\sum_{l=0}^{\infty}\gamma^{l+1}v_\phi(s_{t+l+1})-\sum_{l=0}^{\infty}\gamma^l v_\phi(s_{t+l})\\&=\sum_{l=0}^{\infty}\gamma^l r_{t+l}+\sum_{l=1}^{\infty}\gamma^l v_\phi(s_{t+l})-\sum_{l=0}^{\infty}\gamma^l v_\phi(s_{t+l})\\&=g_t-v_\phi(s_t)\end{aligned} \tag{123}$$

此外，GAE 是优势函数的无偏估计（在 $v_\phi(s_t)$ 准确的前提下），可以对 GAE 求期望验证：

$$
\begin{aligned}
&\mathbb{E}_\pi\left[\hat{A}_t^{\mathrm{GAE}(\lambda),n}(\tau_n) \;\middle|\; S_t=s_t, A_t=a_t\right] \\
&= \frac{1-\lambda}{\lambda-\lambda^{n-t+1}} \underbrace{\sum_{k=1}^{n-t}\lambda^k}_{\frac{1-\lambda^{n-t+1}}{1-\lambda}-1} \left(\underbrace{\mathbb{E}_\pi\left[y_t^k(\tau_n) \mid s_t, a_t\right]}_{Q^\pi(s_t,a_t),\,\forall k} - v_\phi(s_t)\right) \\
&= Q^\pi(s_t,a_t) - v_\phi(s_t) \\
&\overset{!}{=} A^\pi(s_t,a_t)
\end{aligned} \tag{124}
$$

总结一下，**无限时域** GAE 的两个极端情况分别退化到 TD 和 MC 估计，和上节值函数估计中的无限时域 TD(λ) 相互对应。

3.3.3. Asynchronous Advantage Actor-Critic (A3C)

A3C 算法的核心在于异步（asynchronous）。A3C 并行地运行多个环境，每个环境都是一个 A2C，在自己线程里获取和更新全局网络参数，环境之间互不干扰。

异步避免了全局等待，可提高 CPU/GPU 利用率，并且由于多环境并行，按时间顺序自然地会产生多样化的数据，起到了一部分经验重放缓存的作用，即削弱非 i.i.d. 样本之间的相关性。

3.3.4. Convergence of Actor-Critic Methods

若满足满足以下假设，Actor-Critic 方法所得回报通常可以收敛到一个（局部）最大值：
1. MDP 是有限的、遍历的并且奖励有界。
2. actor 和 critic 所用参数化模型是线性的，并使用了恰当的基函数。
3. actor 的学习率低于 critic。
4. critic 所用 TD(λ) 目标的 λ 足够大。

对于深度 Actor-Critic 方法没有特别的保证，但：
1. actor 和 critic 的相对学习率还是很重要。
2. GAE 或 TD(λ) 必须 propagate future reward fast，即要快速将未来奖励的影响传递回来，对应的就是使用较大的 λ（即偏向 MC），早点利用未来奖励的样本，而非慢慢等 $v_\phi(s)$ 等更新。

3.3.5. Continuous Control

Actor-Critic 方法可以处理连续动作 $\boldsymbol{a}\in\mathcal{A}\subset\mathbb{R}^m$，因为它不需要像 DQN 那样通过离散的最大值函数选取策略；且如果 critic 估计值函数而非动作值函数，不依赖动作，在连续动作场景下就不会导致来不及学习、不稳定的问题。

举个例子，可以让策略网络输出分布参数而非动作，比如用对角高斯分布建模策略：

$$
\pi_\theta(\boldsymbol{a}\mid s) \propto \exp\left(-\sum_{i=1}^m \frac{\left(a_i-\mu_\theta(s)_i\right)^2}{2\sigma_\theta(s)_i^2}\right) \tag{125}
$$

网络的输出则是 m 个 head 用于 $\boldsymbol{\mu}_\theta(s)$ 加上 m 个 head 用于 $\boldsymbol{\sigma}_\theta(s)$。

简单情况下，探索采样时可以直接从该策略分布中采样。参数更新时可应用**最大熵正则化**（maximum entropy regularization），定义熵：

$$\mathcal{H}[\pi_\theta] := -\frac{1}{n}\sum_{t=0}^{n-1}\int \pi_\theta(\boldsymbol{a} \mid s_t)\ln \pi_\theta(\boldsymbol{a} \mid s_t)\,\mathrm{d}\boldsymbol{a} \tag{126}$$

其表达了策略 π_θ 在整条轨迹上的平均随机程度。在损失函数中加入正则项：

$$\bar{\mathcal{L}}[\theta] := \mathcal{L}[\theta] - \frac{1}{\beta}\mathcal{H}[\pi_\theta], \quad \beta > 0 \tag{127}$$

最小化该损失函数时也会一定程度上最大化这个熵，从而使得最优化出来的策略仍然具有随机性。在 on-policy 算法中，采样所用的行为策略就是目标策略。最大熵保持了目标策略的一定随机性，也就鼓励了探索的随机性，避免过早收敛到确定策略，导致错过一些状态空间。顺便给出正则项的梯度：

$$\nabla_\theta \mathcal{H}[\pi_\theta] = -\frac{1}{n}\sum_{t=0}^{n-1}\int \pi_\theta(\boldsymbol{a} \mid s_t)\ln \pi_\theta(\boldsymbol{a} \mid s_t)\nabla_\theta \ln \pi_\theta(\boldsymbol{a} \mid s_t)\,\mathrm{d}\boldsymbol{a} \tag{128}$$

3.3.6. Conclusion

如 Figure 3 所示，A2C/A3C 依旧属于随机策略、on-policy 损失函数方法，不同于 REINFORCE 的是又引入了 value-based 的内容（critic 用 target networks 进行值函数估计），往 DQN 方向靠近了一点。

3.4. Trust Region Methods

3.4.1. Off-Policy Gradients

前面提到过，Actor-Critic 等 on-policy 算法的典型问题在于样本利用率低，只能使用当前策略采样的样本进行更新。为了更好地说明这一点，我们试着直接推导 off-policy 梯度，即尝试使用来自另一个策略 μ 的样本来更新当前策略 π_θ。

首先约定：
1. 行为策略 $\mu(a \mid s)$ 满足 $\frac{\pi_\theta(a|s)}{\mu(a|s)} < \infty, \forall a \in \mathcal{A}, \forall s \in \mathcal{S}$。
2. $\xi_t^{\pi_\theta}(s)$ 表示采用策略 π_θ 执行 t 步后状态 s 的分布。

由此之前 REINFORCE 的损失函数梯度可以进一步变化：

$$\begin{aligned}
\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] &= -\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t G_t \nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \\
&= -\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{\pi_\theta}[G_t \mid S_t, A_t]\nabla_\theta \ln \pi_\theta(A_t \mid S_t)\right] \quad \text{(Tower property)} \\
&= -\int \xi_t^{\pi_\theta}(s_t)\int \cancel{\pi_\theta(a_t \mid s_t)}\sum_{t=0}^{n-1}\gamma^t \mathbb{E}_{\pi_\theta}[G_t \mid s_t, a_t]\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\cancel{\pi_\theta(a_t \mid s_t)}}\,\mathrm{d}a_t\,\mathrm{d}s_t \\
&= -\sum_{t=0}^{n-1}\int \xi_t^{\pi_\theta}(s_t)\int \gamma^t Q^{\pi_\theta}(s_t, a_t)\nabla_\theta \pi_\theta(a_t \mid s_t)\,\mathrm{d}a_t\,\mathrm{d}s_t \\
&= -\sum_{t=0}^{n-1}\int \xi_t^{\pi_\theta}(s_t)\int \mu(a_t \mid s_t)\gamma^t Q^{\pi_\theta}(s_t, a_t)\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\mu(a_t \mid s_t)}\,\mathrm{d}a_t\,\mathrm{d}s_t
\end{aligned} \tag{129}$$

这里把期望展开成积分形式和求和形式只是连续和离散的区别，意思差不多。其中 Q^{π_θ} 来自和之前值函数类似的时移性质（Equation 14）：

$$\begin{aligned}\mathbb{E}_{\pi_\theta}[G_t \mid s_t, a_t] &= \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty}\gamma^k R_{k+t} \,\middle|\, S_t = s_t, A_t = a_t\right] \\ &= \mathbb{E}_{\pi_\theta}\left[\sum_{k=0}^{\infty}\gamma^k R_k \,\middle|\, S_0 = s_t, A_0 = a_t\right] \\ &=: Q^{\pi_\theta}(s_t, a_t)\end{aligned} \tag{130}$$

接下来写成采用策略 μ 的轨迹的期望形式（即 $\mathbb{E}_\mu[\cdot] := \mathbb{E}_{\mathrm{T}\sim p^\mu_{\mathrm{T}}(\cdot)}[\cdot]$，与 $\mathbb{E}_\pi$ 对应）：

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] &= -\sum_{t=0}^{n-1}\int \xi_t^\mu(s_t)\frac{\xi_t^{\pi_\theta}(s_t)}{\xi_t^\mu(s_t)}\int \mu(a_t \mid s_t)\gamma^t Q^{\pi_\theta}(s_t, a_t)\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\mu(a_t \mid s_t)}\,\mathrm{d}a_t\,\mathrm{d}s_t \\ &= -\mathbb{E}_\mu\left[\sum_{t=0}^{n-1}\frac{\xi_t^{\pi_\theta}(S_t)}{\xi_t^\mu(S_t)}\gamma^t Q^{\pi_\theta}(S_t, A_t)\frac{\nabla_\theta \pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\right]\end{aligned} \tag{131}$$

接下来要作一些**近似**以便推出可用的形式（TODO），考虑只更新策略分布 $\pi_\theta(\cdot)$，其他例如 $\xi^{\pi_\theta}(\cdot)$、$Q^{\pi_\theta}(\cdot)$、$V^{\pi_\theta}(\cdot)$ 都视作与参数 θ 无关的常数，或者说 `detach` 阻断其对梯度的贡献，记为 $\xi^\pi(\cdot)$ 等。顺便记 $\eta_t(S_t) := \frac{\xi_t^\pi(S_t)}{\xi_t^\mu(S_t)}$，有：

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] &= -\mathbb{E}_\mu\left[\sum_{t=0}^{n-1}\eta_t(S_t)\gamma^t Q^{\pi}(S_t, A_t)\frac{\nabla_\theta \pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\right] \\ &\overset{*}{=} -\mathbb{E}_\mu\left[\sum_{t=0}^{n-1}\eta_t(S_t)\gamma^t (Q^{\pi}(S_t, A_t) - V^\pi(S_t))\frac{\nabla_\theta \pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\right] \\ &= -\nabla_\theta\mathbb{E}_\mu\left[\sum_{t=0}^{n-1}\eta_t(S_t)\gamma^t (Q^{\pi}(S_t, A_t) - V^\pi(S_t))\frac{\pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\right]\end{aligned} \tag{132}$$

其中 $*$ 式的导出参考 Equation 106 中减去 baseline 不影响期望的推导，具体地：

$$\begin{aligned}&\mathbb{E}_\mu\left[\sum_{t=0}^{n-1}\eta_t(S_t)\gamma^t V^\pi(S_t)\frac{\nabla_\theta \pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\right] \\ =&\sum_{t=0}^{n-1}\gamma^t\mathbb{E}_\mu\left[\eta_t(S_t)V^\pi(S_t)\frac{\nabla_\theta \pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\right] \\ =&\sum_{t=0}^{n-1}\gamma^t\int_{s_t}\xi_t^\mu(s_t)\int_{a_t}\cancel{\mu(a_t \mid s_t)}\eta_t(s_t)V^\pi(s_t)\frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\cancel{\mu(a_t \mid s_t)}}\,\mathrm{d}a_t\,\mathrm{d}s_t \\ =&\sum_{t=0}^{n-1}\gamma^t\int_{s_t}\xi_t^\mu(s_t)\eta_t(s_t)V^\pi(s_t)\underbrace{\int_{a_t}\nabla_\theta \pi_\theta(a_t \mid s_t)\,\mathrm{d}a_t}_{0}\,\mathrm{d}s_t \\ =&\,0\end{aligned} \tag{133}$$

按定义 $Q^\pi(S_t, A_t) - V^\pi(S_t)$ 即优势函数 $A^\pi(S_t, A_t)$，此外 $\eta_t(S_t) := \frac{\xi_t^{\pi_\theta}(S_t)}{\xi_t^{\mu}(S_t)}$ 要么难算要么未知模型无法计算，就假设两种策略相近，近似为 1。最终梯度式化为：

$$\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] \approx -\nabla_\theta \mathbb{E}_\mu \left[\sum_{t=0}^{n-1} \gamma^t \frac{\pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)} A^\pi(S_t, A_t) \right] \tag{134}$$

这个式子可以理解为按策略比 $\frac{\pi_\theta(A_t|S_t)}{\mu(A_t|S_t)}$ 进行**重要性采样**（importance sampling），从而通过实际来自 μ 的优势估计 $\hat{A}_t$ 计算假如它来自 π_θ 时会有的样子。

关于重要性采样：

假设有一个函数 $f(X)$，我们希望求其在 $X \sim \pi(\cdot)$ 分布上的期望，即：

$$\mathbb{E}_\pi[f(X)] = \int_x \pi(x) f(x) \, \mathrm{d}x \tag{135}$$

但假如现在我们无法直接从 $\pi(x)$ 上进行采样，只能从另一个分布 $\mu(x)$ 上采样（比如说样本已经是现成的来自 μ 的样本，或者 π 太复杂等情况），那么我们就需要通过来自 μ 的样本间接地获取 π 下的期望：

$$\mathbb{E}_\pi[f(X)] = \int_x \pi(x) f(x) \, \mathrm{d}x = \int_x \mu(x) \frac{\pi(x)}{\mu(x)} f(x) \, \mathrm{d}x = \mathbb{E}_\mu \left[\frac{\pi(x)}{\mu(x)} f(x) \right] \tag{136}$$

这样一来，用 μ 上采集的样本，乘上一个重要性采样比 $\frac{\pi(x)}{\mu(x)}$ 再求期望就可以得到 π 上采集样本时的期望。

如果要用这个 $\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta]$ 尝试实现一个 Actor-Critic 架构的 off-policy 算法，主要要考虑如何去估计这个优势函数。记优势估计为 $\hat{A}_t$（加标记避免和动作 A_t 混淆），一种方式是做类似 Q-Learning 的一步 TD 估计，由：

$$Q^\pi(S_t, A_t) = \mathbb{E}_\mu[R_t + \gamma V^\pi(S_{t+1}) \mid S_t, A_t] \tag{137}$$

将 $V^\pi(s)$ 用估计的 $v_\phi(s)$ 代替，由此得到：

$$\begin{aligned} \nabla_\theta \mathcal{L}_\mu[\theta] &\approx -\nabla_\theta \mathbb{E}_\mu \left[\sum_{t=0}^{n-1} \gamma^t \big(R_t + \gamma v_\phi(S_{t+1}) - v_\phi(S_t) \big) \frac{\pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)} \right] \\ &= -\nabla_\theta \mathbb{E}_\mu \left[\sum_{t=0}^{n-1} \gamma^t \frac{\pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)} \hat{A}_t \right] \end{aligned} \tag{138}$$

当然，这里的优势估计除了这里的 TD 估计，也可以考虑换成之前讨论的 MC、GAE 等形式。实际实现中如果是基于零碎的状态转移样本进行训练（例如 TD 估计）还经常省略会 γ^t，直接使用简洁的 $-\nabla_\theta \mathbb{E}\left[\frac{\pi_\theta(A_t|S_t)}{\mu(A_t|S_t)} \hat{A}_t\right]$，毕竟无所谓连续样本之间的关系了。

在此基础上，我们就可以去复用旧策略 μ 采样的样本了，从而提高样本利用率，加快训练速度。**具体地**，每次都使用当前策略采样获得一系列样本，然后用这些样本多次学习

更新得到新策略，如此往复。但若实际尝试，则会发现样本重复使用次数（repetitions）多了训练就容易不稳定。

3.4.2. Trust-Region Policy Optimization (TRPO)

表现不佳的原因首先是刚刚直接近似为 1 的 $\frac{\xi_t^\pi(S_t)}{\xi_t^\mu(S_t)}$ 项以及重要性采样比 $\frac{\pi_\theta(A_t|S_t)}{\mu(A_t|S_t)}$ 容易数值不稳定。例如，对策略 π_θ 的更新同时改变了状态分布 ξ_t^π，而如果对某个状态 s_t，新策略会有可能采样到而旧策略不会采样到，那么比值就会趋于无穷，反之趋于 0 等。

TRPO 算法的思想很直接，就是围绕旧行为策略 $\mu = \pi_{\theta'}$ 建立一个**信赖域**（trust region），并要求反复使用这部分旧样本更新所得的新策略 π_θ 不允许超出这个信赖域，于是问题变为带约束的优化问题：

$$\min_\theta \mathcal{L}_\mu[\theta] \quad \text{s.t.} \quad \mathbb{E}_\mu\left[\sum_{t=0}^{n-1} D_{\mathrm{KL}}[\mu(\cdot \mid S_t) \parallel \pi_\theta(\cdot \mid S_t)]\right] \le \delta \tag{139}$$

信赖域衡量的是分布之间的“距离”，所以直接使用了 KL 散度（Kullback-Leibler divergence）的概念，它衡量的是用一个近似概率分布 q 来描述真实分布 p 时所引入的信息损失：

$$D_{\mathrm{KL}}(p \parallel q) \coloneqq \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)} \,\mathrm{d}x \tag{140}$$

这个带约束非线性优化问题看着就很棘手，实际实现中可以采用 **Taylor 估计**（Taylor approximation），也就是在当前参数附近做二阶泰勒展开（Hessian），把问题转为**二次约束二次规划**（quadratic constraints quadratic programming，QCQP）问题。

每次更新，当前（旧）策略为 μ，参数位于 θ_μ，在此处有梯度 $\boldsymbol{g} \coloneqq \nabla_\theta \mathcal{L}_\mu[\theta]\big|_{\theta=\theta_\mu}$ 和 Hessian 矩阵 $\boldsymbol{H}$。于是损失函数和约束分别近似为：

$$\mathcal{L}_\mu[\theta] \approx \boldsymbol{g}^\top(\theta - \theta_\mu), \quad D_{\mathrm{KL}}[\mu \parallel \pi_\theta] \approx \frac{1}{2}(\theta - \theta_\mu)^\top \boldsymbol{H}(\theta - \theta_\mu) \tag{141}$$

求解上述优化问题得：

$$\theta^* = \theta_\mu + \alpha\sqrt{\frac{2\delta}{\boldsymbol{g}^\top \boldsymbol{H}^{-1}\boldsymbol{g}}}\boldsymbol{H}^{-1}\boldsymbol{g} \tag{142}$$

这个解称为**自然策略梯度**（natural policy gradient）。此外，由于我们是近似得到的，所以照这样更新还是有可能违反 KL 约束，所以式中的 α 就是 TRPO 在此基础上添加的系数，可以通过**线搜索**（line search）去找到尽可能大的符合约束的 α，具体地，比如说先试 $\alpha = 1$，不对就尝试乘一个因子缩小一点再尝试，如此往复直到符合约束。

3.4.3. Proximal Policy Optimization (PPO)

在 TRPO 上再稍加修改就是经常使用的**近端策略优化**（proximal policy optimization，PPO）算法了。

如前所述，TRPO 的约束还是太复杂了，虽然可以二阶近似求解但还是有点麻烦，或许可以更简单一点。考虑 $\mathcal{L}_\mu[\theta]$ 中的策略比 $\frac{\pi_\theta(A_t|S_t)}{\mu(A_t|S_t)}$，我们尝试要求它只能卡（“clip”）到 $1 \pm \epsilon$

的范围里，这样也能确保旧策略和新策略不要离太远，但比 KL 散度约束要更简单粗暴。具体地，采用如下和损失函数：

$$\mathcal{L}_{\mu}^{\text{clip}}[\theta] := -\mathbb{E}_{\mu}\left[\sum_{t=0}^{n-1}\gamma^t \min\left\{\frac{\pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}\hat{A}_t, \text{clip}\left(\frac{\pi_\theta(A_t \mid S_t)}{\mu(A_t \mid S_t)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right\}\right] \tag{143}$$

观察式子，它其实是在截断和不截断所得两种 loss 中选择了较小的一个。考虑优势估计的符号，分类讨论一下会发现当策略比超出范围时，实际也只有两种情况会采用截断后的信号：

1. $\hat{A}_t > 0$ 且 $\frac{\pi_\theta(A_t|S_t)}{\mu(A_t|S_t)} > 1+\epsilon$ 时。
2. $\hat{A}_t < 0$ 且 $\frac{\pi_\theta(A_t|S_t)}{\mu(A_t|S_t)} < 1-\epsilon$ 时。

这两种情况分别是“动作好还想大幅提高该动作概率”以及“动作不好还想大幅度降低该动作概率”，换句话说就是只在 loss 会变好时截断以阻止过大幅度的策略改进。那么截断能干什么？截断处就是把 loss 削平了，于是梯度也就为零，从而让这部分优势不参与参数更新。

与之前会不稳定的直接 off-policy 梯度更新法比较，PPO 随着样本重复使用次数提高显著加速训练，同时稳定性也不错，不过在动作集规模 $|\mathcal{A}|$ 较大时容易不稳定。

回到谱图 Figure 3，TRPO/PPO 算法依旧是随机策略、采用 Actor-Critic 架构估计值函数的算法。此外章节初提到过，TRPO/PPO 虽然涉及样本复用，但我们还是将其划分为 on-policy 算法；图中的划分方式倒也可以理解，因为基本的损失函数 $\mathcal{L}_{\mu}[\theta]$ 确实是基于 off-policy 思想推导的，所以称为“off-policy loss”，只不过 TRPO/PPO 在新旧策略分布上做了一些约束以确保没有“off”得太远。

关于算法原理与工程实现的距离：

TRPO 到 PPO 表现的提升大多不是这些算法层面的裁剪带来的，实际上主要依靠 PPO 实现中的一系列工程技巧。毕竟从原理层面上来说，PPO 只是为了降低复杂度实现的对 TRPO 的近似，按理说效果应该

具体可以查看一篇 Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO（https://arxiv.org/pdf/2005.12729），其中分析了 PPO 标准实现中应用的一系列技巧并做了详细的消融实验。如果将这些技巧应用到 TRPO 上，在某些任务上其性能也可能超越 PPO。还有 https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/。

不止 PPO，绝大部分算法实践中的优秀效果都离不开大量工程技巧的应用，完全按算法朴素地实现通常效果不会太好。

3.4.4. Group Relative Policy Optimization (GRPO)

强化学习算法也可以用于大语言模型（large language model，LLM）的微调。对于一个预训练语言模型，可以将其推理视为一种预测下一个单词的强化学习策略：

$$\pi_\theta(a_t \mid s_0, a_0, ..., a_{t-1}) \tag{144}$$

再加上信赖域机制限制策略更新幅度以防其遗忘预训练知识，此外还需要训练一个奖励模型提供奖励函数，毕竟基本没法手动为这么大的状态空间设计奖励。如果采用 PPO 算法，critic 需要学习一个值函数估计 $v_\phi(s)$，但这东西将会需要和 LLM 规模相当的参数量，不现实，并且状态空间过大、维度过高很难保证泛化性能。

考虑 Group Relative Policy Optimization（GRPO）算法，对于一次询问 s_0 运行生成 G 条不同轨迹 $\{s_n^i\}_{i=1}^G$，得到 G 个最终奖励 $r(s_n^i)$。用这一整组奖励的均值替代 $v_\phi(s_0)$ 作为对值函数的估计：

$$\nu(s_0) := \frac{1}{G}\sum_{i=1}^{G} r(s_n^i) \approx V^{\pi_\theta}(s_0) \tag{145}$$

优势估计也改用 $\nu(s)$ 和奖励来近似：

$$\hat{A}_t^i := \underbrace{r_t^i}_{\approx 0} + \underbrace{\gamma}_{1}\underbrace{v_\phi(s_{t+1}^i)}_{\approx r(s_n^i)} - \underbrace{v_\phi(s_t^i)}_{\approx \nu(s_0)} \approx \frac{r(s_n^i) - \nu(s_0)}{\mathrm{std}\left(\{r(s_n^i)\}_{i=1}^G\right)} \tag{146}$$

除以标准差保证尺度归一化。很粗暴的近似，直接表达最终奖励相对平均奖励的优势。

GRPO 的表现取决于与任务的适配性，和 PPO 没有绝对的高下之分，不适合效果就可能很差。此外，通常较小的 G 稳定性表现更优。

4. Off-Policy Actor-Critic

TODO

4.1. Deterministic Policy Gradients (DPG)

之前的策略都是采用随机概率分布建模，动作来自对随即策略的采样，现在考虑**确定策略**（deterministic policy），用一个确定的函数映射 $\boldsymbol{\pi}_\theta : \mathcal{S} \to \mathcal{A}$ 建模。考虑连续动作 $\boldsymbol{a} \in \mathcal{A} \subset \mathbb{R}^m$，critic 用参数化模型 $Q_\phi(s_t, \boldsymbol{a}_t)$ 估计值函数 $Q^\pi(s_t, \boldsymbol{a}_t)$，基于之前 off-policy 策略梯度推导的某个中间形式进行近似有：

$$\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] \approx -\nabla_\theta \sum_{t=0}^{n-1} \int_{s_t} \xi_t^\mu(s_t) \int_{\boldsymbol{a}_t} \gamma^t \underbrace{Q^\pi(s_t, \boldsymbol{a}_t)}_{\approx Q_\phi(s_t, \boldsymbol{a}_t)} \pi_\theta(\boldsymbol{a}_t \mid s_t) \, \mathrm{d}\boldsymbol{a}_t \, \mathrm{d}s_t \tag{147}$$

而确定策略就相当于某个动作概率为 1 的随机策略（即有 $\pi_\theta(\boldsymbol{\pi}_\theta(s) \mid s) = 1, \forall s$），故可简化积分得：

$$\begin{aligned}
\nabla_\theta \mathcal{L}_{\pi_\theta}[\theta] &\approx -\nabla_\theta \sum_{t=0}^{n-1} \int_{s_t} \xi_t^\mu(s_t) \gamma^t Q_\phi(s_t, \boldsymbol{\pi}_\theta(s_t)) \, \mathrm{d}s_t \\
&= -\nabla_\theta \sum_{t=0}^{n-1} \int_{s_t} \xi_t^\mu(s_t) \gamma^t Q_\phi(s_t, \boldsymbol{\pi}_\theta(s_t)) \, \mathrm{d}s_t \\
&= -\nabla_\theta \sum_{t=0}^{n-1} \gamma^t \mathbb{E}_\mu \left[Q_\phi(S_t, \boldsymbol{\pi}_\theta(S_t)) \right] \\
&\propto -\nabla_\theta \mathbb{E}_{t \sim p(\cdot \mid \gamma) \propto \gamma^t} \left[\mathbb{E}_\mu \left[Q_\phi(S_t, \boldsymbol{\pi}_\theta(S_t)) \right] \right]
\end{aligned} \tag{148}$$

最后一步是将 t 视为服从分布 $p(t \mid \gamma) \propto \gamma^t$ 的一个离散随机变量，从而将式子化为期望形式。原本需要在完整轨迹样本上进行计算，如此变化后就可以在数据集上对 t 采样来进行计算，特别是针对以状态转移形式存储的数据（transitions），可以在每条数据中存储其在所在轨迹中的时间步 t 来实现计算。不过实践中有时由于 γ 接近 1、数据不包含时间戳 t 等原因会省略 γ^t。

4.1.1. Deep DPG (DDPG)

类似 DQN 之于 Q-Learning，使用深度神经网络作为 DPG 的参数化模型，并加入一系列如经验重放缓存、目标网络和软目标更新、Batch-norm over inputs（BN）、加入噪声随机探索等提高稳定性和表现的技术，得到 DDPG。

要实现 DPG 需要维护两个网络（不算目标网络），一个是作为确定策略的 actor 网络 $\boldsymbol{\pi}_\theta(s)$，一个是作为动作值函数估计的 critic 网络 $Q_\phi(s, \boldsymbol{a})$。二者分别优化各自的损失函数，actor 优化：

$$\arg\min_\theta -\mathbb{E}_\mu \left[\sum_{t=0}^{n-1} \gamma^t Q_\phi(S_t, \boldsymbol{\pi}_\theta(S_t)) \right] \tag{149}$$

同时 critic 优化：

$$\arg\min_{\phi} \mathbb{E}_{\mu}\left[\sum_{t=0}^{n-1}\left(R_t + \gamma Q_{\phi'}(S_{t+1}, \boldsymbol{\pi}_{\theta'}(S_{t+1})) - Q_{\phi}(S_t, S_t)\right)^2\right] \tag{150}$$

用 PyTorch 实现大致如下：

```python
from torch.optim import Adam
from torch.nn.functional import mse_loss

actor_optimizer = Adam(actor.parameters())
critic_optimizer = Adam(critic.parameters())
actor_t, critic_t = deepcopy(actor), deepcopy(critic)

for _ in range(max_updates):
    batch = self.replay_buffer.sample()

    # Critic update
    q_values = critic(batch['states'], batch['actions'])
    next_actions = actor_t(batch['next_states'])
    targets = batch['rewards'] + gamma * (~batch['terminals'] \
              * critic_t(batch['next_states'], next_actions))
    critic_optimizer.zero_grad()
    mse_loss(q_values, targets.detach()).backward()
    critic_optimizer.step()

    # Actor update
    q_values = critic(batch['states'], actor(batch['states']))
    actor_optimizer.zero_grad()
    (-q_values * batch['discounts']).mean().backward()
    actor_optimizer.step()

    # Target network updates
    actor_t = target_model_updates(actor_t, actor)
    critic_t = target_model_updates(critic_t, critic)
```

其中 `actor_t` 和 `critic_t` 是目标网络（target network），类似之前在 DQN 的示例中的使用，`.detach()` 阻断目标网络的梯度更新，`target_model_updates` 中可以具体实现目标网络的同步方式（如软更新）。这里的数据中用 `batch['discounts']` 存储了各条 transition 对应的 γ^t。

两个网络需要分别使用两个优化器进行优化，因为优化策略的 actor loss 中包含值函数网络 $\boldsymbol{\pi}_\theta$，值函数网络的参数不应该受到策略更新的影响，我们不希望在 actor 优化时同时影响两个网络的参数。

4.1.2. Twin Delayed DDPG (TD3)

类似之前对 Double Q-Learning 的讨论，DPG 也有一样的过估计问题：

$$\mathbb{E}\left[\max_{\theta} Q(s, \boldsymbol{\pi}_\theta(s))\right] \geq \max_{\theta} \mathbb{E}[Q(s, \boldsymbol{\pi}_\theta(s))] \tag{151}$$

TD3 是表现最好的 DDPG 变体之一，其对 DDPG 作了一些补充。首先是 TD3 中的 “Twin”，引入 Clipped Double Q-Learning（CDQ），考虑维护两个 critic Q 网络，但和 Double Q-Learning 一个选取最优动作一个评估价值（构造目标）的思路不同，TD3 中这两个网络是平等的，使用时取其中较小的值，所以其实是一种保守估计方法，或者说 pessimistic 的。

第二是 regularization with clipped noise，在更新 Q 网络时最小化 loss：

$$\mathcal{L}_i^Q[\phi_i] = \mathbb{E}_\mu \left[\sum_{t=0}^{n-1} \left(R_t + \gamma \min_{j \in \{1,2\}} Q_{\phi'_j}(S_{t+1}, \pi_{\theta'}(S_{t+1}) + \epsilon) - Q_{\phi_i}(S_t, S_t) \right)^2 \right] \tag{152}$$

其中 $\epsilon \sim \text{clip}(\mathcal{N}(\mathbf{0}, \boldsymbol{\sigma}^2), -\boldsymbol{c}, \boldsymbol{c}) \in \mathbb{R}^{|\mathcal{A}|}$，除了多了一个 Q 网络，区别就在于在策略上加了一个噪声（clip 以防太大），以平均掉一些确定策略下可能的单点尖峰错误的影响。

第三是 TD3 中的 “Delayed”，critic 和 actor 不再同频率更新，而是 critic 更新多次后才更新一次 actor。这是由于 actor 的 loss 依赖 critic 的值函数估计，此举尝试让 critic 估计更稳定后再更新 actor。

在谱图 Figure 3 中，DDPG 和 TD3 属于 off-policy 确定策略算法，critic 估计动作值函数，并且采用 pessimism 估计稳定 Q 值估计。

4.2. Robust Decision Making

TODO

4.2.1. Regularization

4.2.2. Robust Reinforcement Learning

4.2.3. Planning as Inference

4.2.4. Stochastic Policy Gradients (SPG) and Deterministic Policy Gradients (DPG)

4.2.5. Reparameterization Trick

4.3. Maximum Entropy RL

4.3.1. Soft Actor-Critic (SAC)

SAC 可以基于 TD3 的框架实现，具体地：1、使用重放缓冲、pessimism，但不用 clipped noise；2、

5. Exploration

探索（exploration）是数据的来源，之前我们已经使用过一些探索技巧，例如 **ϵ-贪婪策略**，即以 ϵ 的概率（一般会随探索逐渐下调直至一个下限）随机选择动作，其余情况按目标策略采样：

$$\pi_\theta^\epsilon(a \mid s) \coloneqq (1-\epsilon)\pi_\theta(a \mid s) + \epsilon\frac{1}{|\mathcal{A}|} \tag{153}$$

还有基于价值函数的 **Boltzmann 探索**（uninformed Boltzmann exploration）：

$$\pi_\theta^\beta(a \mid s) \coloneqq \frac{\exp(\beta Q_\theta(s,a))}{\sum_{a'}\exp(\beta Q_\theta(s,a'))} \tag{154}$$

当 $\beta = 1$ 时，就是用 softmax 函数把价值函数表示成了动作概率分布；当 $\beta \to 0$ 时，各动作概率趋于相似，整体策略趋于纯随机探索；当 $\beta > 1$ 时，本身价值函数较大的动作权重也更大，策略趋于只选择最大 Q 值动作的确定性策略。在 Equation 126 还提到过**最大熵正则化**，基于损失函数，保持策略的随机性。

优化的探索策略是相对纯随机探索来说的，那么怎样的探索策略算是更智能的？一个自然的选择是优先选择已经观察到高回报的动作，即围绕当前最优策略（exploitation policy）进行探索。毕竟过去的经验认为某个动作价值比较高，那么之后也可以更频繁地采用它。前述 ϵ-贪婪策略和 Boltzmann 探索策略都属于这一类，在当前最优下引入少量随机性。这类策略也天生存在缺陷，容易过快收敛于短期奖励指示的最优奖励（前期容易达到的奖励），并不断巩固这一策略最终错过可能存在的长期高回报。

另一种是从广泛探索的角度出发，优先选择回报确定性不高的动作。以基于价值函数估计的算法为例，价值函数估计的准确性依赖于在这部分状态空间上探索的充分性；回报估计波动大，说明探索不充分。特别是对于 Q-Learning 这样的 off-policy 算法，优先探索还没有被充分探索、评估的区域可以加速价值函数估计的收敛，辅助智能体更好地做出长期判断。不过相较值迭代等算法中可以通过判断“更新后价值函数是否还改变”来确定是否已经收敛，这类策略是通过回报样本来衡量不确定性的。这就导致如果系统本身就有随机性，称为**偶然不确定性**（aleatoric uncertainty），这类策略就容易陷进去出不来。具体一点，比如说系统在某处给予回报本来就是随机的（比如投个骰子决定回报），再多的探索也不会降低这里回报的不确定性，而这类探索策略还是会义无反顾地选择它，无法脱身。

既然客观的偶然不确定性无法降低（irreducible），那么就让智能体更主观地评估“不确定性”，优先选择还没有尝试过的动作，就不会陷入循环。动作是否已经尝试过完全是从智能体的视角来说的，未尝试过的动作表示其对应状态空间尚未被完全探索，这种不确定性可称为**认知不确定性**（epistemic uncertainty），不会像偶然不确定那样因系统性质导致无法降低。但这类策略也有问题，一是难以处理连续动作空间的情况，二是会导致过度探索——本来智能的策略就是希望用有限的算力探索过于庞大的状态空间中的低维有效信息，现在这样就有退回到暴力搜索的意思了。

5.1. Uncertainty

前面提到了无法降低的偶然不确定和可以降低的认知不确定性，都属于各种因素导致的不确定性（uncertainty）。除此之外，如果是基于模型的方法，错误的模型也会导致不可降低的模型偏差（model bias）。

均方误差是非常常用的损失函数，假设在数据分布 $\mathcal{D}$ 上计算，$X \sim \rho(\cdot), Y \sim \mathcal{N}(\cdot \mid \mu(X), \sigma^2(X))$，我们可以将 MSE 分解为几个部分：TODO

$$\begin{aligned} & \underbrace{\mathbb{E}_{\mathcal{D}}\Big[\mathbb{E}\Big[(Y - f_{\mathcal{D}}(x))^2 \;\Big|\; \mathcal{D}\Big]\Big]}_{\text{generalization error}} \\ = & \underbrace{\mathbb{E}[\sigma^2(X)]}_{\text{aleatoric}} + \underbrace{\mathbb{E}[\mathbb{V}_{\mathcal{D}}[f_{\mathcal{D}}(X) \mid X]]}_{\text{epistemic}} + \underbrace{\mathbb{E}\Big[(\mathbb{E}_{\mathcal{D}}[f_{\mathcal{D}}(X) \mid X] - \mu(X))^2\Big]}_{\text{model-bias}} \end{aligned} \tag{155}$$

TODO

5.2. Thompson Sampling

5.3. Optimistic Exploration

6. Offline Reinforcement Learning

TODO

7. Multi-Agent Reinforcement Learning

TODO

8. Multi-Task Reinforcement Learning

TODO

9. Model-Based Reinforcement Learning

TODO